



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master Degree in Computer Science
Curriculum Artificial Intelligence

Hebbian Co-Design of Morphing Winged Drones

Supervisors:

Prof. Andrea Cossu

Andrea Vicari, PhD Cand.

Dr. Fuda Van Diggelen

Prof. Dario Floreano

Candidate:

Salvatore Mastrangelo

ACADEMIC YEAR 2025/2026

Dedica

Glossary

- **PCA:** Principal Component Analysis
- **BPTT:** Backpropagation Through Time
- **LSTM:** Long Short-Term Memory Networks
- **MDP:** Markov Decision Process
- **POMDP:** Partially Observable Markov Decision Process
- **RL:** Reinforcement Learning
- **PPO:** Proximal Policy Optimization
- **AC-RL:** Actor-Critic Reinforcement Learning
- **EA:** Evolutionary Algorithms
- **ES:** Evolution Strategy
- **CMA-ES:** Covariance Matrix Adaptation Evolution Strategy
- **NSGA-II:** Non-dominated Sorting Genetic Algorithm II
- **EPANN:** Evolved Plastic Artificial Neural Network
- **GPU:** Graphics Processing Unit
- **URDF:** Unified Robot Description Format
- **IR:** Intermediate Representation

Contents

1	Introduction	6
2	Technical Background & Related Work	7
2.1	Deep Reinforcement Learning	7
2.1.1	Markov Decision Processes	7
2.1.2	Actor-Critic Reinforcement Learning	9
2.1.3	Proximal Policy Optimization	11
2.2	Long Short-Term Memory Networks	12
2.2.1	Recurrent Neural Networks	12
2.2.2	Vanishing Gradient Problem	12
2.2.3	Long Short-Term Memory	12
2.3	Evolutionary Algorithms	14
2.3.1	CMA-ES	15
2.3.2	NSGA-II	17
2.4	Hebbian Learning	19
2.4.1	Hebb's Postulate	19
2.4.2	The Generalized Hebbian Rule	20
2.4.3	Plasticity Rules as the Object of Optimization	21
2.5	Related Work	22
2.5.1	Co-Design of Morphology and Control	22
2.5.2	Evolved and Learned Plasticity	24
2.5.3	Positioning of This Thesis	26
3	Problems	27
3.1	The Sequential Design Problem	27
3.2	The Morphology Optimization Problem	28
3.3	The Computational Feasibility Problem	28
4	DA RILEGGERE Simulation Environment	31
4.1	Genesis: The Simulator	31
4.1.1	Architecture and Taichi Kernels	31
4.1.2	Advantages and Disadvantages	33

4.1.3	Physics Simulation	34
4.1.4	Aerodynamics Simulation	37
4.1.5	Simulation Noise	41
4.2	Flight Environment	42
4.2.1	Uniform Density Forests	43
4.2.2	Growing Density Forests	44
4.2.3	Latin Hypercube Forests	46
4.3	Drone Model	48
4.3.1	Drone Shape and Actuators	48
4.3.2	Sensors	50
4.3.3	Drone Genome	51
4.3.4	The Standard Drone	53
5	Methodology	56
5.1	Base Controllers	56
5.1.1	Specialist LSTM Controller	56
5.1.2	Generalist LSTM Controller	56
5.1.3	Training the Generalist LSTM Controller	56
5.2	The Hebbian Controller	56
5.2.1	Hebbian Rules on the Output Layer	56
5.2.2	Weight Decay and Reset Semantics	56
5.3	Inner Loop: Evolving the Hebbian Rules	56
5.4	Outer Loop: Evolving the Drone Morphology	56
5.5	The Full Co-Design Pipeline	56
6	Results and Experiments	57
6.1	Inner Loop Experiments	57
6.1.1	Recovering Performance on Generalist Controllers	57
6.1.2	Improving Performance on Specialist Controllers	57
6.2	Full Co-Design Experiments	57
6.2.1	Co-Design vs. Morphology-Only Evolution	57
6.2.2	Swapping the Pareto Fronts	57
6.2.3	PCA Analysis of the Evolved Morphologies	57
6.2.4	Morphology Choices of The Evolution	57
6.2.5	Generalization Capabilities of Evolved Controllers	57
6.2.6	Convergence or Plasticity?	57
7	Conclusions & Future Work	58
7.1	Conclusions	58
7.2	Future Work	58

7.2.1	Meta-Learning Techniques	58
7.2.2	Testing on Real Drones	58
7.2.3	Testing on Different Systems	58

Introduction

Technical Background & Related Work

This chapter introduces the main techniques used in this work, and a series of related works that have influenced the development of the approach proposed in the following chapters. Each of the methods presented here, in addition to some novelties presented in the following chapters, is a building block of the final system.

2.1 Deep Reinforcement Learning

A common approach to train a deep neural network controller is by using Reinforcement Learning (RL): an agent interacts with an environment through a sequence of actions, and the only training signal it receives is a scalar *reward* that grades how well it is performing. RL rests on the *reward hypothesis*: that any goal can be expressed as the maximization of expected cumulative reward [46].

2.1.1 Markov Decision Processes

At each discrete time step t , the agent observes the state $s_t \in \mathcal{S}$ of the environment and selects an action $a_t \in \mathcal{A}$; the environment responds with a new state s_{t+1} and a scalar reward r_{t+1} , computed by the environment's own, generally unknown, transition and reward functions.

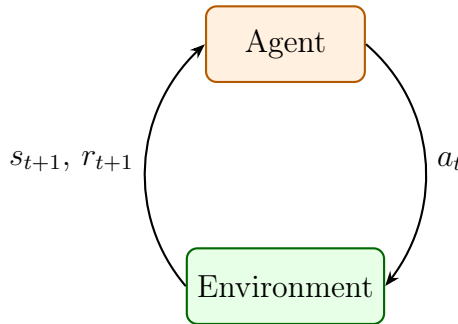


Figure 2.1: The agent-environment interaction loop. The reward r_{t+1} is a scalar with no fixed sign convention beyond larger being better: it is the only feedback the agent receives beyond a partial, possibly noisy observation of the environment.

This loop is assumed to be *Markovian*: the next state and reward depend only on the current state and action, not on the full history that produced them,

$$P(s_{t+1}, r_{t+1} \mid s_t, a_t) = P(s_{t+1}, r_{t+1} \mid s_0, a_0, r_1, \dots, s_{t-1}, a_{t-1}, r_t, s_t, a_t). \quad (2.1)$$

The Markov property is what lets the current state summarize everything relevant about the past, and is what makes the following formalism, a Markov Decision Process (MDP), tractable.

Definition 2.1.1 (Markov Decision Process). *A Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where:*

- $\mathcal{S}$ is the set of states,
- $\mathcal{A}$ is the set of actions,
- $P(s' | s, a) = P(s_{t+1}=s' | s_t=s, a_t=a)$ is the state-transition distribution,
- $R(s, a) = \mathbb{E}[r_{t+1} | s_t=s, a_t=a]$ is the expected immediate reward,
- $\gamma \in [0, 1]$ is a discount factor.

The agent's behaviour is described by a policy π , a map from states to actions: a *deterministic* policy is a function $a = \pi(s)$, while a *stochastic* policy $\pi(a | s) = P(a_t=a | s_t=s)$ is a conditional distribution over actions. Stochasticity is the source of the exploration needed to discover good actions in the first place, since a purely deterministic agent can never try anything it does not already favour. The RL objective is to find a policy that maximizes the *return*, the total discounted future reward from a given time step,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (2.2)$$

The discount factor trades off immediate against delayed reward: $\gamma \approx 0$ makes the agent myopic, $\gamma \approx 1$ far-sighted; besides this qualitative role, $\gamma < 1$ is also what keeps G_t finite for a task with no fixed episode length and bounded per-step reward.

Given a policy π , the *state-value function* $v_\pi(s)$ and *action-value function* $q_\pi(s, a)$ are the expected return obtained from a state, or from a state-action pair, when following π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | s_t=s], \quad q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t=s, a_t=a]. \quad (2.3)$$

Because the return decomposes into an immediate reward and a discounted continuation, $G_t = r_{t+1} + \gamma G_{t+1}$, the value functions satisfy a recursive consistency condition, the *Bellman expectation equation*:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) v_\pi(s') \right) = \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a). \quad (2.4)$$

An *optimal* policy π_* is one whose action-value function $q_*(s, a) = \max_\pi q_\pi(s, a)$ dominates that of every other policy in every state; since some deterministic policy always achieves q_* , namely $\pi_*(s) = \arg \max_a q_*(s, a)$, finding q_* is by itself sufficient to solve the MDP.

When $\mathcal{S}$ and $\mathcal{A}$ are small and finite and (P, R) are fully known, v_π and q_* can be computed exactly, by dynamic-programming methods such as policy iteration or value iteration, which apply the Bellman equations above as update rules until convergence (the *model-based* setting). In the *model-free* setting considered for the rest of this section, which includes every task in this work, P and R are unknown, and value functions must instead be estimated from sampled transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ obtained by actually interacting with the environment. Whenever $\mathcal{S}$, or $\mathcal{A}$, is continuous or otherwise too large to enumerate (as is the case for the sensor observations

and continuous control commands used throughout this work), a lookup table, as in tabular Q-Learning [52], is no longer an option, and v_π , q_π , or π itself must instead be represented by parametric function approximators, typically deep neural networks; this is what is meant by *deep* reinforcement learning. The remainder of this section develops the *policy-based* branch of deep RL, which parametrizes the policy directly rather than deriving it from a value function, and which is the branch this work builds on.

2.1.2 Actor-Critic Reinforcement Learning

Policy-based methods parametrize the policy itself, $\pi_\theta(a | s)$, a deep neural network with parameters θ , and optimize such parameters directly by gradient ascent on the expected return

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]. \quad (2.5)$$

Compared to deriving a policy from a value function, this tends to have better convergence properties and remains effective in high-dimensional or continuous action spaces. Its cost is that $J(\theta)$, and its gradient, can only be evaluated through samples, which tends to be high-variance and can converge to a local rather than a global optimum [46].

The *policy gradient theorem* [47] shows that, despite $J(\theta)$ depending on the environment’s unknown transition dynamics through the states π_θ itself visits, its gradient can be written without any explicit knowledge of those dynamics:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a | s) q_{\pi_\theta}(s, a)]. \quad (2.6)$$

Intuitively, $\nabla_\theta \log \pi_\theta(a | s)$ is the direction in parameter space that most increases the probability of action a in state s ; weighting it by $q_{\pi_\theta}(s, a)$ pushes θ to raise the probability of actions that lead to high return and lower that of actions that lead to low return.

Actor-critic methods reduce this variance by learning a second, separate approximation of the value function (the *critic*, e.g. $V_w(s) \approx v_{\pi_\theta}(s)$) alongside the policy, or *actor*, π_θ [27]. The critic is trained as in any value-based method, by a temporal difference (TD) update towards $r_{t+1} + \gamma V_w(s_{t+1})$, while the actor is updated by policy gradient ascent, using the critic’s own estimate of value in place of the true, unknown q_{π_θ} .

Figure 2.2 depicts both networks reading the same state s_t for simplicity, but nothing in the algorithm requires this symmetry: since only the actor π_θ is kept for deployment while the critic V_w is discarded once training ends, the critic is free to observe a wider, *privileged* state that the actor never sees: quantities that are cheap to obtain in simulation but unavailable, or unreliable, on the deployed system. Because such an asymmetric critic only ever shapes the actor’s parameter updates and never appears in its computation graph at test time, this cannot leak information the actor could not otherwise access; it simply gives the critic a less noisy value estimate to train against, which in turn lowers the variance of the advantage used to update the actor.

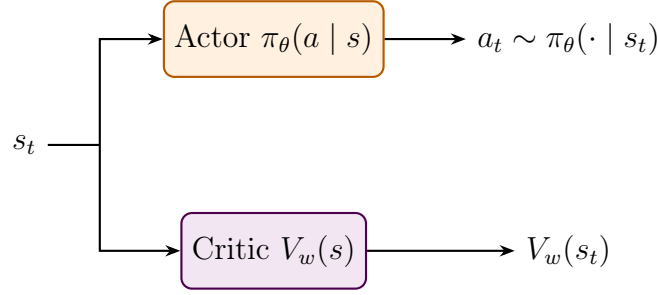


Figure 2.2: The actor-critic architecture: two networks read the same state s_t , the actor to sample an action, the critic to estimate its value $V_w(s_t)$. The TD error between $V_w(s_t)$ and $r_{t+1} + \gamma V_w(s_{t+1})$, observed once the environment responds, is what trains the critic by regression and reweights the actor’s policy-gradient update.

A further variance reduction method consists in subtracting a state-dependent baseline from the critic’s estimate without introducing any bias: since, for any fixed s , $\mathbb{E}_{a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a | s)] = 0$, replacing $q_{\pi_\theta}(s, a)$ in the policy gradient with the *advantage function*

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s) \quad (2.7)$$

leaves $\nabla_\theta J(\theta)$ unchanged in expectation while typically reducing its variance substantially, since $A_\pi(s, a)$ measures how much better action a is than the policy’s own average behaviour in s , rather than the absolute, and generally much larger, magnitude of the return itself. Computing $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$ exactly would still require two separate value functions, but the single TD error $\delta_t = r_{t+1} + \gamma V_w(s_{t+1}) - V_w(s_t)$ is already, on its own, a one-sample estimate of the advantage: it substitutes the critic’s own V_w for $v_\pi(s_t)$, and the sampled $r_{t+1} + \gamma V_w(s_{t+1})$ for $q_\pi(s_t, a_t)$.

Relying on a single δ_t , however, looks only one step ahead, so the estimate inherits whatever bias V_w currently has; summing several consecutive TD errors looks further into the future and averages more of that bias away, at the cost of compounding sampling variance the further the sum reaches. Generalized Advantage Estimation (GAE) [41] controls this trade-off directly, combining successive TD errors into an exponentially-weighted sum governed by a decay parameter $\lambda \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}. \quad (2.8)$$

GAE interpolates between the high-bias, low-variance one-step estimate $\hat{A}_t = \delta_t$ ($\lambda=0$) and the low-bias, high-variance Monte Carlo estimate obtained by summing every future TD error ($\lambda=1$). This is the same bias-variance trade-off, controlled by the same kind of exponentially-decaying trace, that $\text{TD}(\lambda)$ eligibility traces implement for the value function itself [46]. The actor and critic are typically two networks, or two heads of a partly shared network, trained jointly end to end: the critic by regression towards its TD target, the actor by advantage-weighted policy gradient ascent.

2.1.3 Proximal Policy Optimization

The actor-critic parameter update above does not, by itself, say how large a step to take. Each batch of collected experience is only informative about the policy $\pi_{\theta_{\text{old}}}$ that generated it, yet the gradient estimate computed from it is noisy; an update that moves θ too far from θ_{old} because of the strength of that noisy estimate can push π_{θ} into a region of the parameter space the batch says nothing reliable about, collapsing performance in a way that plain gradient ascent has no mechanism to prevent. Trust Region Policy Optimization (TRPO) [40] addresses this directly, by constraining each update to a region, measured by KL divergence between π_{θ} and $\pi_{\theta_{\text{old}}}$, within which the current batch is trusted to be informative, at the cost of a second-order constrained-optimization step that is expensive computationally.

Proximal Policy Optimization (PPO) [42] achieves a similar effect with an unconstrained, first-order objective, optimizable directly by ordinary stochastic gradient ascent, which is what has made it, in practice, a default choice for on-policy deep RL. Let

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2.9)$$

be the probability ratio between the policy being optimized and the fixed, older policy that generated the batch (so that $r_t(\theta_{\text{old}}) = 1$), and let $\hat{A}_t$ be an advantage estimate, e.g. GAE. The plain policy-gradient surrogate $\mathbb{E}_t[r_t(\theta)\hat{A}_t]$ has the correct gradient at $\theta = \theta_{\text{old}}$, but nothing stops $r_t(\theta)$ from drifting arbitrarily far from 1 once optimized repeatedly on the same batch. PPO instead maximizes the *clipped surrogate objective*

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.10)$$

where ϵ , typically between 0.1 and 0.2, sets the width of the trust region. When $\hat{A}_t > 0$, the clip caps the objective's growth once $r_t(\theta)$ exceeds $1 + \epsilon$, removing any incentive to keep increasing $\pi_{\theta}(a_t | s_t)$ further on this particular batch; symmetrically, when $\hat{A}_t < 0$, it caps the objective once $r_t(\theta)$ falls below $1 - \epsilon$. The outer minimum makes L^{CLIP} a pessimistic (lower) bound on the unclipped surrogate, so clipping only ever activates in the direction that would otherwise make the update too aggressive, never in the direction that would make it too conservative. In practice, this first-order clipping behaves much like TRPO's explicit trust region, at a fraction of its implementation and computational cost.

The actor and critic are trained jointly, by combining L^{CLIP} with a value-function regression loss for the critic, $L^{\text{VF}}(\theta) = (V_{\theta}(s_t) - (\hat{A}_t + V_{\theta_{\text{old}}}(s_t)))^2$, and an entropy bonus $S[\pi_{\theta}](s_t)$ that discourages the policy from collapsing prematurely onto a deterministic one, into a single objective

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 S[\pi_{\theta}](s_t), \quad (2.11)$$

maximized by a handful of epochs of minibatch stochastic gradient ascent over each freshly collected batch of trajectories [42]. Once these epochs are done the batch is discarded, $\theta_{\text{old}} \leftarrow \theta$ is refreshed, and a new batch is collected with the updated policy: an on-policy training loop that strictly alternates environment interaction under a fixed policy with optimization of that same policy on the data it just produced.

2.2 Long Short-Term Memory Networks

2.2.1 Recurrent Neural Networks

Since the state-describing vectors are often inaccessible to a controller, and the only accessible knowledge is the sequence of observations, such settings are formally *Partially Observable Markov Decision Processes* (POMDPs) [26]. A recurrent neural network (RNN) bridges this gap by maintaining an internal state that captures information about the past. It processes a sequence $x_1, \dots, x_T$, that might represent a time series or a sequence of observations itself, and maintains a state h_t that is updated at every step from the current input and the previous state [11]:

$$h_t = g(Uh_{t-1} + Wx_t + b), \quad h_t \in \mathbb{R}^{d_h}, \quad x_t \in \mathbb{R}^{d_{in}}, \quad U \in \mathbb{R}^{d_h \times d_h}, \quad W \in \mathbb{R}^{d_h \times d_{in}}, \quad (2.12)$$

with g a pointwise nonlinearity (typically $\tanh$). Unrolled over time, this is a feed-forward computation with the same weights U, W shared across every step, trained end-to-end by backpropagation through time (BPTT) [53]: a forward pass computes every h_t in temporal order, and a backward pass propagates gradients from the last step back to the first, through the same recurrent weight matrix U at every step.

2.2.2 Vanishing Gradient Problem

This repeated multiplication by U is also the vanilla RNN's main weakness. Informally, the gradient of the loss at step T with respect to an early state h_t scales, over the $T - t$ intervening steps, with a product of $T - t$ Jacobians of the state-transition function, each dominated by U ; if the dominant eigenvalue of U is below 1 this product shrinks geometrically with $T - t$ (*vanishing gradient*), and if it is above 1 it grows geometrically (*exploding gradient*) [36]. In the vanishing case, the influence of inputs far in the past disappears from the state before training can pick up on it, which makes vanilla RNNs difficult to train on long-range dependencies. A first fix is to make the state update a convex combination, $h_t = \alpha h_{t-1} + (1 - \alpha) \tanh(Uh_{t-1} + Wx_t)$, so that a fraction α of the state is carried over unchanged instead of being fully overwritten at every step; the LSTM (below) can be read as turning this fixed, hand-set α into a learned, input-dependent, per-dimension quantity.

2.2.3 Long Short-Term Memory

Long Short-Term Memory (LSTM) networks [23] address the vanishing/exploding gradient problem by keeping two separate pieces of recurrent state instead of one, and by making the recurrence of the long-term one additive rather than a repeated matrix multiplication:

- the *hidden state* $h_t \in \mathbb{R}^{d_h}$ plays the same role as the state of a vanilla RNN: it is what is read out for prediction or further processing at step t ;
- the *cell state* $c_t \in \mathbb{R}^{d_h}$ is an internal memory that carries information across time-steps, and is only ever combined with the rest of the network through

an elementwise multiplication and an elementwise addition, never a matrix product. This is what keeps the gradient with respect to c_t from vanishing or exploding as it is backpropagated through time.

What is written into c_t , what is dropped from it, and what part of it is exposed through h_t is decided at every step by three *gates*.

Definition 2.2.1 (Gate). *A gate is a standard affine layer of h_{t-1} and x_t followed by a sigmoid activation, used to modulate a signal by elementwise multiplication:*

$$\gamma_t = \sigma(U_\gamma h_{t-1} + W_\gamma x_t + b_\gamma) \in (0, 1)^{d_h}, \quad (2.13)$$

so that, for any signal $v \in \mathbb{R}^{d_h}$, $\gamma_t \odot v$ lets a component of v through unchanged where $\gamma_t \approx 1$ and suppresses it where $\gamma_t \approx 0$.

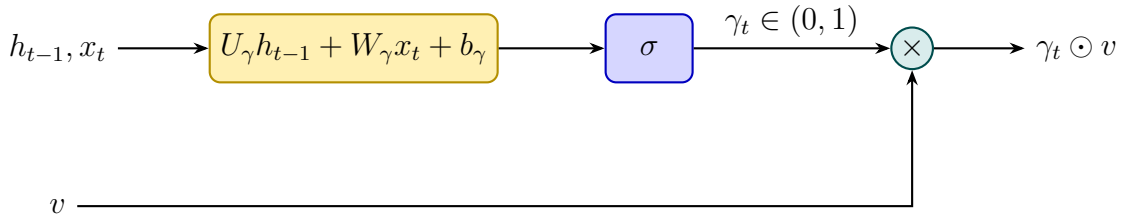


Figure 2.3: A gate: an affine map of the previous hidden state and the current input, squashed to $(0, 1)$ by a sigmoid, used to modulate a signal v by elementwise multiplication.

An LSTM cell uses three such gates, each with its own parameters:

- the **forget gate** f_t decides which part of the old cell state c_{t-1} is kept;
- the **input gate** i_t decides which part of a candidate update g_t (computed exactly as a vanilla RNN state would be) is written into the cell state;
- the **output gate** o_t decides which part of the cell state is exposed as the new hidden state h_t .

$$f_t = \sigma(U_f h_{t-1} + W_f x_t + b_f) \quad \text{forget gate} \quad (2.14)$$

$$i_t = \sigma(U_i h_{t-1} + W_i x_t + b_i) \quad \text{input gate} \quad (2.15)$$

$$g_t = \tanh(U_g h_{t-1} + W_g x_t + b_g) \quad \text{candidate cell content} \quad (2.16)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad \text{cell state update} \quad (2.17)$$

$$o_t = \sigma(U_o h_{t-1} + W_o x_t + b_o) \quad \text{output gate} \quad (2.18)$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{hidden state update} \quad (2.19)$$

where $\odot$ denotes the elementwise product. Each gate, and the candidate g_t , has its own weight matrices $U_\square$, $W_\square$ and bias $b_\square$, learned jointly with the rest of the network: a single LSTM layer of hidden size d_h therefore holds four sets of parameters shaped like a single vanilla-RNN layer's, one each for f_t , i_t , g_t , o_t .

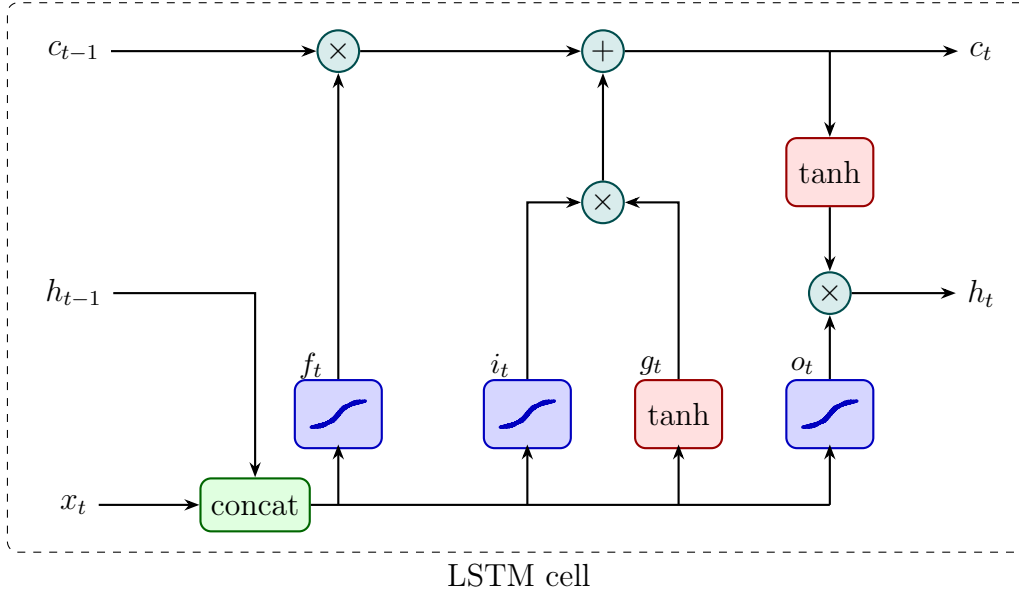


Figure 2.4: Structure of an LSTM cell. The top rail carries the cell state c_t across time-steps through only an elementwise multiplication (forget gate f_t) and an elementwise addition (gated candidate $i_t \odot g_t$), with no matrix product in between: this is the property that keeps gradients from vanishing or exploding along this path. The hidden state h_t is a gated, squashed read-out of c_t , and h_{t-1}, x_t jointly drive all four gates/candidate at the bottom.

Figure 2.4 makes explicit why this design addresses the problem laid out in the previous section: the path carrying c_t across time-steps involves only elementwise operations gated by f_t and $i_t \odot g_t$, with no repeated multiplication by a shared weight matrix. Backpropagating through this path therefore does not force the gradient to contract or expand geometrically with the number of time-steps, which is what allows an LSTM to learn dependencies over much longer sequences than a vanilla RNN.

2.3 Evolutionary Algorithms

Evolutionary Algorithms (EAs) are a family of population-based, gradient-free optimization methods loosely inspired by natural selection [10]. Where a gradient method follows the slope of the objective from a single point, an EA maintains a whole *population* of candidate solutions, or *individuals*, each encoded by a vector of parameters called its *genome*, and repeats a simple loop over successive *generations*:

1. *Evaluation*: every individual is assigned a *fitness*, a scalar score obtained by actually running it on the task;
2. *Selection*: the fitter individuals are chosen as *parents*, so that good genomes get to reproduce more than bad ones;
3. *Variation*: new individuals, the *offspring*, are produced from the parents by

mutation (a random perturbation of a single genome) and *recombination*, or *crossover*, which mixes the genomes of two parents;

4. *Replacement*: the offspring form the next population, possibly alongside the best individuals of the current one (*elitism*), so that a good solution, once found, is never lost.

Over many generations, selection pressure concentrates the population in high-fitness regions of the search space, while variation keeps exploring around it.

The appeal of this recipe is that it treats the objective as a black box: nothing is assumed about it beyond the ability to evaluate it. It needs no gradient, no differentiability, and no smoothness, and it is indifferent to whether the genome is continuous, discrete, or mixed. This makes EAs the natural tool whenever the quantity being optimized is only defined through a complete simulation and, more generally, whenever gradient-based training, such as the RL methods of the previous section, cannot reach the parameters of interest. The price is sample efficiency: every generation costs as many full evaluations as there are individuals, which is affordable mainly when evaluations can be run in parallel, as is the case for GPU-batched simulation. Two EAs are used in this work, one for each of the two settings they were designed for: CMA-ES for single-objective optimization over continuous parameters, and NSGA-II for problems with several competing objectives.

2.3.1 CMA-ES

Evolution Strategies (ES) are the branch of EAs specialized to continuous search spaces, $x \in \mathbb{R}^n$. In an ES, variation is not a discrete crossover but a Gaussian perturbation: at every generation, λ offspring are sampled around a single mean vector m ,

$$x_k = m + \sigma z_k, \quad z_k \sim \mathcal{N}(0, C), \quad k = 1, \dots, \lambda, \quad (2.20)$$

where $\sigma > 0$ is a global *step size* and $C \in \mathbb{R}^{n \times n}$ is a covariance matrix that shapes the cloud of samples. The offspring are evaluated and sorted by fitness, and the mean is moved to a weighted average of the best $\mu < \lambda$ of them,

$$m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda}, \quad w_1 \geq w_2 \geq \dots \geq w_{\mu} > 0, \quad \sum_{i=1}^{\mu} w_i = 1, \quad (2.21)$$

where $x_{i:\lambda}$ denotes the i -th best of the λ samples. Note that only the *ranking* of the fitness values enters the update, never their magnitude: the algorithm is therefore invariant to any monotone rescaling of the objective, and requires no normalization of the reward. The flip side is that, when the fitness is a noisy estimate (as the outcome of a stochastic simulation is), what degrades the update is precisely noise large enough to reorder the candidates, and the standard remedies are to enlarge λ or to average each individual's fitness over repeated evaluations.

With σ and C fixed, this is a very simple algorithm, whose performance depends entirely on whether the cloud of samples happens to match the local shape of the fitness landscape: a narrow, tilted valley, for instance, forces an isotropic Gaussian to take tiny steps, since most large steps fall off the valley walls. The Covariance

Matrix Adaptation Evolution Strategy (CMA-ES) [20, 19] removes this dependence by adapting both σ and C online, from the history of the steps that turned out to be successful, so that the sampling distribution itself learns the shape of the landscape; Figure 2.5 illustrates the effect.

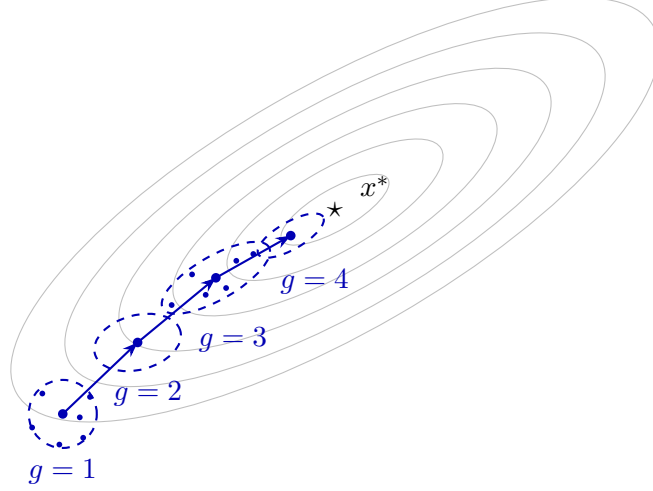


Figure 2.5: Covariance adaptation in CMA-ES on a fitness landscape shaped like a narrow, tilted valley (grey contours; the optimum is x^*). Each dashed ellipse is the one-standard-deviation contour of the sampling distribution $\mathcal{N}(m, \sigma^2 C)$ at generation g , and the dots are its samples. The search starts isotropic ($C = I$), but the successful samples of each generation lie preferentially along the valley, and the covariance update progressively stretches the distribution in that direction: by $g = 3$ the algorithm is taking long steps along the valley and short ones across it, which an isotropic search could never do.

The covariance is updated, after every generation, towards the directions in which the selected offspring were found. Writing $y_{i:\lambda} = (x_{i:\lambda} - m)/\sigma$ for the normalized step that produced the i -th best sample, the update reads

$$C \leftarrow (1 - c_1 - c_\mu) C + c_1 p_c p_c^\top + c_\mu \sum_{i=1}^{\mu} w_i y_{i:\lambda} y_{i:\lambda}^\top, \quad (2.22)$$

with small learning rates $c_1, c_\mu \in (0, 1)$. The last term is the weighted empirical covariance of this generation's successful steps: it stretches the distribution along the directions the good samples came from, and shrinks it along the others. The middle term does the same across generations, through the *evolution path* p_c , an exponentially-decaying running sum of the successive displacements of the mean: a direction in which m keeps moving, generation after generation, is amplified, while directions in which it merely oscillates cancel out. On a smooth objective the net effect is that C comes to approximate the inverse Hessian near the optimum, so that CMA-ES behaves much like a second-order method, without ever computing a derivative.

The step size is adapted separately, by *cumulative step-size adaptation*: a second evolution path accumulates the displacements of the mean, rescaled by $C^{-1/2}$ so that, if selection were purely random, its length would be that of a random walk. If

consecutive steps point in a consistent direction the path grows longer than a random walk would, which signals that the steps are too short and σ is increased; if they largely cancel out, the path is shorter than expected and σ is decreased. This lets the algorithm take large steps while far from the optimum and contract progressively as it closes in on it. The learning rates, the weights w_i , and the population size $\lambda = 4 + \lfloor 3 \ln n \rfloor$ all have default values that work robustly across problems, so that the number of parameters the user must provide is reduced to just the initial mean $m^{(0)}$ and step size $\sigma^{(0)}$: this near absence of tuning is a large part of why CMA-ES is the default choice for black-box optimization in moderate dimensions.

2.3.2 NSGA-II

Many design problems do not have a single objective. A plane should travel far *and* consume little energy; a structure should be light *and* stiff. Typically such objectives conflict, so that improving one worsens the other, and no single solution is best in every respect. *Multi-objective optimization* makes this explicit by asking to minimize a vector of M objectives at once, $f(x) = (f_1(x), \dots, f_M(x))$, where an objective to be maximized is handled simply by negating it. Since there is no natural total order among vectors, the notion of “better” has to be replaced by a partial one:

Definition 2.3.1 (Pareto dominance). *A solution x dominates a solution x' , written $x \prec x'$, if it is no worse in every objective and strictly better in at least one:*

$$f_i(x) \leq f_i(x') \quad \forall i \in \{1, \dots, M\} \quad \text{and} \quad f_j(x) < f_j(x') \quad \text{for some } j. \quad (2.23)$$

The representation of such dominance can be visualized in a two-dimensional space in Figure 2.6(a). A solution that is dominated by no other is *Pareto-optimal*; the set of all such solutions is the *Pareto set*, and its image in objective space is the *Pareto front*. Every point on the front is a trade-off that cannot be improved in one objective without giving something up in another, and which of them is “best” is not a question the optimizer can answer: it is left to the designer, once the front is known. The goal of a multi-objective optimizer is therefore not one point but an approximation of the whole front: a set of solutions that lies close to it (*convergence*) and is spread evenly along it (*diversity*). One could obtain this by fixing a weighted sum of the objectives and running a single-objective optimizer once per choice of weights, but that requires choosing the weights in advance, and a weighted sum cannot reach the points of a front where it bends inwards (its non-convex parts). A population-based method, in contrast, already maintains a set of solutions, and can be steered to spread that set along the front within a single run; this is why EAs are the dominant approach to multi-objective optimization.

The Non-dominated Sorting Genetic Algorithm II (NSGA-II) [6] is the most widely used such method. It is a genetic algorithm in the sense of section 2.3: for real-valued genomes, the standard variation operators are simulated binary crossover (SBX) and polynomial mutation [5], both of which produce offspring in the neighbourhood of their parents. What distinguishes it is the selection step, which is built on two ingredients, both illustrated in Figure 2.6(b).

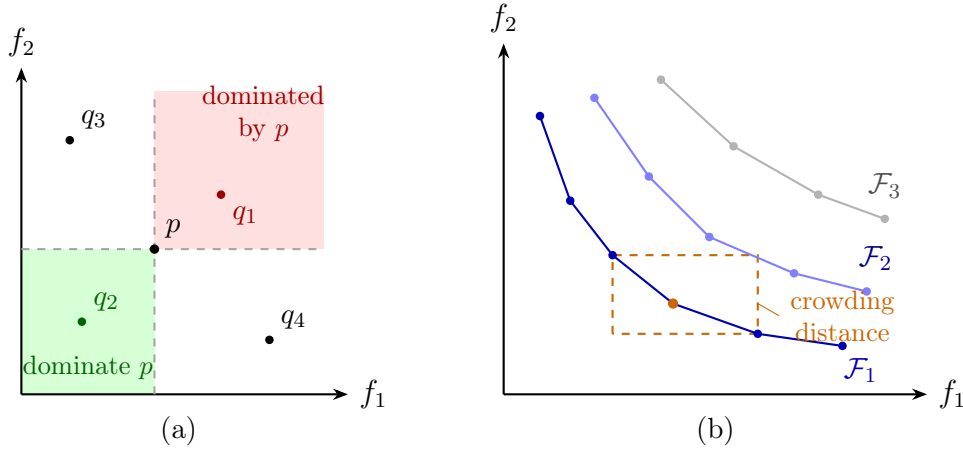


Figure 2.6: Two objectives, both to be minimized. (a) Pareto dominance: p dominates every point in the shaded red region (q_1), is dominated by every point in the green one (q_2), and is *incomparable* with points such as q_3 and q_4 , each of which is better than p in one objective and worse in the other. (b) Non-dominated sorting of a population into fronts $\mathcal{F}_1 \prec \mathcal{F}_2 \prec \mathcal{F}_3$: $\mathcal{F}_1$ contains the points dominated by no other, $\mathcal{F}_2$ those dominated only by members of $\mathcal{F}_1$, and so on. The dashed box is the crowding cuboid of the orange point of $\mathcal{F}_1$: the rectangle spanned by its nearest neighbours along each objective, whose side lengths sum to its crowding distance. A point with a large cuboid sits in a sparsely populated part of its front, and is preferred over one in a crowded region.

The first is *non-dominated sorting*, which partitions the population into a sequence of *fronts* $\mathcal{F}_1, \mathcal{F}_2, \dots$: $\mathcal{F}_1$ is the set of individuals that no other individual dominates, $\mathcal{F}_2$ the set of those that become non-dominated once $\mathcal{F}_1$ is removed, and so on until every individual has been assigned to a front. The index of an individual's front is its *rank*, and lower is better: $\mathcal{F}_1$ is the current approximation of the Pareto front, $\mathcal{F}_2$ the second-best layer behind it. The “fast” procedure of NSGA-II computes this partition in $O(MN^2)$ time for a population of N individuals, which is negligible next to the cost of evaluating them.

The second is the *crowding distance*, which breaks ties among individuals of the same front in favour of diversity. For each objective, the front is sorted along that objective and each individual is assigned the distance between its two neighbours, normalized by the objective's range; the crowding distance is the sum of these values over all objectives, and the two extreme individuals of each front, which have only one neighbour, receive an infinite value so that they are always kept. Geometrically, it measures the size of the largest box around an individual that contains no other member of its front: a large value marks an isolated individual in a sparsely covered region, which is exactly where the front's approximation most needs improving.

The two are combined into a single *crowded-comparison* rule: between two individuals, the one with the lower rank wins, and if the ranks are equal, the one with the larger crowding distance does. A generation of NSGA-II then proceeds as follows. Parents are chosen from the population P of size N by binary tournaments under this rule, and produce N offspring Q by crossover and mutation. Parents and

offspring are merged into a pool $P \cup Q$ of size $2N$, the pool is sorted into fronts, and the next population is filled with whole fronts in order, $\mathcal{F}_1$ first, until the next front no longer fits; that last front is truncated to the remaining slots by keeping its most isolated members, in descending order of crowding distance. Merging parents and offspring makes the algorithm elitist (a non-dominated individual can only be displaced by one that dominates it or by a more isolated peer), while the crowding distance keeps the population spread along the front without any additional parameter to tune, which is the combination that made NSGA-II the reference algorithm for problems with two or three objectives.

Since the output of a multi-objective run is a set rather than a number, comparing two runs, or tracking a single run across generations, requires a scalar measure of front quality. The most common is the *hypervolume* indicator [54]: the volume of objective space (an area, when $M = 2$) that is dominated by the front and bounded by a fixed *reference point*, chosen to be worse than every solution in every objective. A front that lies closer to the true Pareto front, or covers a wider portion of it, dominates a larger region, so the hypervolume rewards convergence and diversity at once; it is only meaningful, however, when the same reference point is used for every front being compared.

2.4 Hebbian Learning

Hebbian learning is a different, and much older, view of what learning in a neural network can be: a synaptic weight changes as a consequence of the activity that flows through it, according to a *local* rule that involves no error signal and needs no supervision, and keeps doing so for as long as the network is running.

2.4.1 Hebb's Postulate

The idea originates in neuropsychology. Hebb [21] proposed, as the mechanism by which associations form in the brain, that

When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased,

a statement usually compressed into the slogan “*cells that fire together wire together*”. The postulate says nothing about what the strengthened connection is *for*: the synapse strengthens simply because the two neurons it links have repeatedly been active at the same time.

For a rate-based neuron model (the one used by artificial neural networks), the simplest formalization considers a single postsynaptic unit with output y , receiving presynaptic activities $x \in \mathbb{R}^n$ through a weight vector $w \in \mathbb{R}^n$, $y = w^\top x$, and lets each weight change in proportion to the product of the activities at its two ends:

$$\Delta w_j = \eta x_j y, \quad j = 1, \dots, n, \quad (2.24)$$

with $\eta > 0$ a small *learning rate*. For a whole layer with input $x \in \mathbb{R}^n$, output $y \in \mathbb{R}^m$ and weight matrix $W \in \mathbb{R}^{m \times n}$, the same rule reads, in matrix form, as an outer product:

$$\Delta W = \eta y x^\top, \quad \Delta W_{ij} = \eta y_i x_j. \quad (2.25)$$

The dominant property of (2.24), shared by the whole family of rules that descends from it, is *locality*. The update of W_{ij} depends only on x_j and y_i , the two activities physically present at the synapse, and on nothing else: not on the activity of other neurons, not on the state of other layers, and not on any measure of how well the network is doing. By contrast, with backpropagation the gradient of a loss L with respect to the same weight, $\partial L / \partial W_{ij} = \delta_i x_j$, has the same presynaptic factor x_j but replaces the postsynaptic activity y_i with an error term δ_i that is computed at the output of the network and transported backwards through every downstream layer (Figure 2.7). A Hebbian update can therefore be applied at every forward pass, during the normal operation of the network.

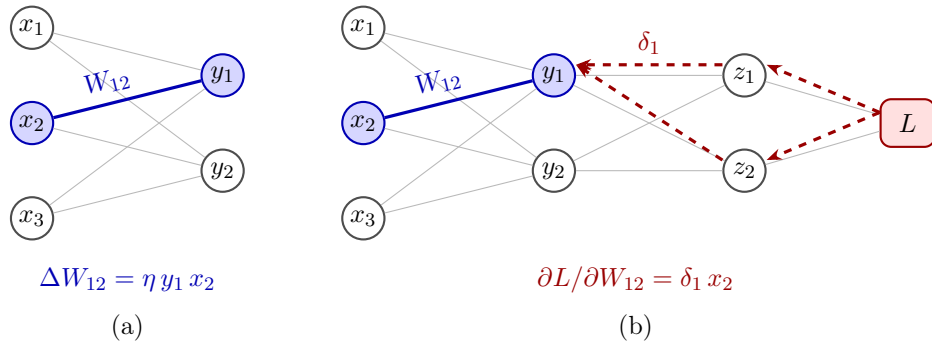


Figure 2.7: Locality of the Hebbian update. (a) Under Hebb’s rule, the change of the highlighted weight W_{12} is a function of the two activities at its ends, x_2 and y_1 , and of nothing else; it can be computed in the forward pass, at the synapse. (b) Under backpropagation, the same weight’s gradient keeps the presynaptic factor x_2 but replaces y_1 with an error δ_1 , which only exists once a loss L has been evaluated at the output and its gradient carried back (dashed) through every layer downstream of the synapse.

2.4.2 The Generalized Hebbian Rule

A local rule is any function $F(\cdot)$ of the quantities available at the synapse:

$$\Delta W_{ij} = \eta F(x_j, y_i, W_{ij})$$

Gerstner and Kistler [16] note that, for small activities, any such F can be expanded in a Taylor series around $x_j = y_i = 0$, and that truncating the expansion after the bilinear term gives the most general rule that is at most linear in each activity:

$$\Delta W_{ij} = \eta (A_{ij} x_j y_i + B_{ij} x_j + C_{ij} y_i + D_{ij}), \quad (2.26)$$

usually referred to as the *ABCD rules* [35, 45]. Each coefficient has a distinct meaning:

- A is the *correlation* term, Hebb’s rule itself. A positive A strengthens a synapse when its two ends are simultaneously active, a negative one weakens it, which decorrelates the neuron’s output from that input;
- B is the *presynaptic* term: the weight changes whenever the input is active, regardless of what the neuron does. With $B < 0$, an input that keeps firing without contributing to the output is progressively weakened, the “pre active, post silent” form of suppression;
- C is the *postsynaptic* term: the weight changes whenever the neuron fires, regardless of that particular input. With $C < 0$, every synapse of a neuron is weakened each time it fires, a normalization that redistributes strength from the inputs that stay silent to those that actually drive it;
- D is a constant drift, independent of activity: a steady creep of the weight in one direction, which can act as a bias, or, with the opposite sign to the weight, as a crude decay.

It should be noted that in case of repeated activations, the rule may lead to a runaway effect, where the weights grow without bound. This is particularly true for the A term, which can cause a positive feedback loop if the inputs are consistently active. To overcome this problem, various normalization techniques can be applied, such as weight clipping or weight decay. While weight clipping simply limits the maximum value a weight can take, weight decay introduces a small penalty to the weight values, effectively reducing them over time, in a fashion similar to regularization in supervised learning:

$$\Delta W_{ij} = \eta (A_{ij} x_j y_i + B_{ij} x_j + C_{ij} y_i + D_{ij}) - \lambda W_{ij}$$

The coefficients can be shared by an entire layer, as one rule for all the synapses, or, as written in (2.26), be a separate matrix each, of the same shape as W , so that every synapse follows its own rule [35, 34]. In the latter case a layer of $m \times n$ weights is described by $4mn$ coefficients, and the learning rate η may itself be per synapse. If η is not fixed but modulated by a dedicated signal, this is called *neuromodulation* [45]. The point of this parametrization is that the same template, with different coefficients, produces qualitatively different behaviours: a pure correlation detector, a decorrelating one, a synapse that only ever decays, one that potentiates and then forgets. Which of them is appropriate at a given synapse is not something the rule can decide, since it has no access to the task; the coefficients are the knobs through which a purpose is imposed on a rule that is otherwise blind to it.

2.4.3 Plasticity Rules as the Object of Optimization

In a *plastic* network, the weights are no longer the parameters that define the network: they are part of its state, initialized to some value at the beginning of a *lifetime*, either an episode or the deployment itself, and driven from then on by the rules, along a trajectory that depends on the inputs the network happens to receive. What stays fixed across lifetimes, and is therefore what actually defines the

network, are the rules, the rate η and the initial weights. The consequence is that the same network, placed in two different environments, ends up with two different sets of weights, each shaped by the activity that environment induced. This is the mechanism by which a plastic network can adapt, within a lifetime and without any gradient, to conditions that were not present when it was designed, and at the same time the reason a rule is not a solution: it is a procedure, which only becomes useful once its coefficients are such that the weight trajectory it produces improves the network's behaviour rather than degrading it.

Finding such coefficients is an optimization problem, with an unusual objective: the quality of a rule is the performance of the network at the end of, or throughout, a lifetime of applying it, which depends on the coefficients only through the whole sequence of weight updates and of the outputs those weights produced. This is a form of *meta-learning* in which the outer level searches over learning rules and the inner level is the lifetime of the network under one set of them. The objective can in principle be differentiated through the chain of updates [31], but when the lifetime is a closed-loop interaction with an environment, as in control, the dependence of the outcome on the coefficients is long, indirect, and only defined by running it. In such cases it is possible to treat them as a black box and search them with the evolutionary algorithms of section 2.3. Networks obtained this way, whose genome encodes rules rather than weights, are known as *evolved plastic artificial neural networks*, or EPANNs [14, 44].

2.5 Related Work

This thesis sits at the meeting point of two lines of research that have so far been pursued separately: the joint optimization of a robot's body and of its controller, and the design of neural controllers whose weights keep changing during operation under rules that are themselves the object of optimization. This section reviews the works of each line that are closest to what is done here, singling out in each case the design choice that the present work makes differently, and closes by stating where it stands with respect to both.

2.5.1 Co-Design of Morphology and Control

Designing a robot means choosing a body and a controller, and the two choices are not independent: the best controller depends on the body it drives, and the value of a body can only be judged through the best behaviour it affords. *Co-design* makes this coupling explicit by optimizing both. Writing m for a morphology in a design space $\mathcal{M}$, θ for the parameters of a controller and $F(m, \theta)$ for the performance of the closed loop they form, obtained by simulating it, the problem is a nested one:

$$m^* = \arg \max_{m \in \mathcal{M}} F(m, \theta^*(m)), \quad \theta^*(m) = \arg \max_{\theta} F(m, \theta). \quad (2.27)$$

The formulation states the problem without solving it: F is a black box, and evaluating the outer objective at a single candidate m requires solving, or approximating, the inner problem for that body. The works reviewed below differ mainly in how they pay for the inner level, and (2.27) is a convenient way to compare them.

The earliest approach collapsed the two levels into one. Sims [43] evolved virtual creatures whose genome, a directed graph, encoded at the same time the segments of the body and the neural circuits embedded in each segment, obtaining swimmers, walkers and jumpers from a single genetic algorithm; Lipson and Pollack [28] applied the same idea to machines made of bars, actuators and artificial neurons, and fabricated the evolved designs by rapid prototyping. In both cases the pair (m, θ) is a single individual and the inner problem is never solved explicitly: the controller of a body is whatever the mutations happened to pair with it.

Cheney et al. [4] identified why this becomes a problem as the design space grows. A controller is co-adapted to the body it evolved with, so a mutation of the body is almost always harmful in the short term, however promising the new body may be once a controller is newly adapted to it. Selection therefore removes morphological novelty faster than it can be exploited, and the search converges prematurely on a body found early. Their remedy, *morphological innovation protection*, temporarily lifts selection pressure from individuals whose body has just changed, giving the controller time to catch up before the body is judged. The underlying principle carries over to every method in this section: a body can only be assessed fairly once a controller has been adapted to it.

Eiben and Hart [9] turned that principle into a design rule, arguing that every newborn body should go through a learning phase before its fitness is measured. Gupta et al. [17] showed what this buys at scale. In their Deep Evolutionary Reinforcement Learning framework, bodies are kinematic trees of cylinders connected by hinge joints, a space of the order of 10^{18} morphologies; an asynchronous tournament evolution mutates limbs and joints, and the controller of every new body is trained from scratch by PPO (subsection 2.1.3) on the body's own egocentric sensors for a fixed budget of interactions, with the fitness of the body being the reward reached at the end of training. Two findings are relevant here. Bodies evolved in harder environments, uneven terrain and manipulation rather than flat ground, perform better when transferred to new tasks, which the authors read as environmental complexity fostering morphological intelligence. And although nothing selects for it explicitly, the time a controller needs to reach a given performance halves within ten generations: evolution finds bodies that are easier to control, the morphological counterpart of the Baldwin effect. What makes these results possible is also what makes the method expensive: the inner problem of (2.27) is paid in full at every evaluation, with a complete reinforcement learning run per body and no inheritance of the controller, which required more than a thousand CPUs running in parallel.

The same division of labour has recently been applied to drones. Muff et al. [33] evolve hexacopters in which each of the six arms is described by its length, two placement angles, two motor tilt angles and the spin direction of its rotor. A $(\mu + \lambda)$ evolution strategy with tournament selection acts on these morphological parameters, with repair operators that resolve collisions between arms and a static hovering test that discards unviable designs, while the controller of each morphology, a small feed-forward network, is learned from scratch by PPO on four waypoint-following tasks (circle, slalom, shuttle run and figure of eight). The evolved drones, with tilted rotors and asymmetric arms, outperform the conventional hexacopter on every task. The authors also introduce metrics for the learning process itself and

find that evolved bodies reach their learning breakthrough earlier and converge to higher rewards, confirming the Baldwin-like effect of Gupta et al. As in that work, evolution owns the morphology and reinforcement learning owns the controller, and the cost of the inner level, several hundred million simulation steps per body, is what limits the search to under a thousand morphologies.

Bergonti et al. [3], in the laboratory where this thesis was carried out, address a morphing winged drone of the same family as the one used in the present work, a fixed-wing glider whose wings carry actuated joints. The outer level is NSGA-II (subsection 2.3.2) over the wing geometry, the number and orientation of the morphing joints of each wing, the choice of servomotors and propulsion unit from commercial catalogues, and a scalar that weighs the two objectives, energy consumption and mission time over a suite of scenarios. The inner level is not a learned controller but a trajectory optimization: for every candidate design, a nonlinear program solved by direct multiple shooting computes the time series of joint torques and thrust that completes each scenario at minimum cost, subject to the multi-body dynamics with aerodynamic forces, the actuator limits, the obstacles and the checkpoints. The co-designed drones cut energy by more than a third and mission time by a fifth or more with respect to the commercial glider they start from. The choice of an open-loop optimal trajectory as inner solution is what makes tens of thousands of inner evaluations affordable on CPUs, but it assumes a known model and no disturbances, and it leaves the closed-loop controller that would fly the real drone outside the optimization.

A different way to escape the cost of the inner level is to amortize it. Gupta et al. [18] train a single Transformer policy, MetaMorph, on a large set of bodies at once, encoding each limb as a token that combines its proprioceptive readings with a description of its geometry and joints, so that the policy reads the morphology it is controlling as one more input modality. The result generalizes zero-shot to unseen variations of kinematics and dynamics and transfers to new bodies with little additional training. In the terms of (2.27), $\theta^*(m)$ is replaced by a single θ for a policy $\pi(a \mid o, m)$ conditioned on m : the inner problem is paid once, for the whole design space, at the price of telling the controller which body it has.

2.5.2 Evolved and Learned Plasticity

The idea of letting evolution choose the learning rule rather than the weights, introduced in subsection 2.4.3, also has a history in the laboratory where this work was done. Floreano and Urzelai [14] evolved controllers for a Khepera robot in which the genome specified, for each synapse, which of four Hebbian rules it obeyed and at what rate, while the weights themselves were drawn at random at the beginning of every life and shaped only by the rule. On a task that required a sequence of behaviours, these controllers not only learned it within a life but, once evolved in simulation, transferred to the physical robot, and to a larger robot with a different body, better than controllers with evolved fixed weights. The review by Soltoggio, Stanley and Risi [44] consolidates two decades of such EPANNs and draws from them one lesson that matters for the present work: plasticity is selected only when something varies, between or within lifetimes, that a fixed genome cannot antici-

pate. On a stationary task evolution finds it cheaper to write the solution into fixed weights, and a plastic mechanism that is never needed is at best neutral and often harmful. Neuromodulation, the gating of the learning rate by a dedicated signal, is the other recurring theme of that literature [45].

Miconi, Stanley and Clune [31] took the opposite route and made the plasticity differentiable. Each connection carries a fixed weight W_{ij} and a Hebbian trace H_{ij} whose contribution is scaled by a coefficient α_{ij} :

$$y_i = \sigma\left(\sum_j (W_{ij} + \alpha_{ij} H_{ij}) x_j\right), \quad H_{ij} \leftarrow (1 - \eta) H_{ij} + \eta x_j y_i, \quad (2.28)$$

and W , α and η are all trained by gradient descent, backpropagating through the entire lifetime of the network, trace updates included. The decomposition is instructive: the weight is split into a part that is learned once and a part that is written by experience and forgotten geometrically, and α decides, synapse by synapse, how much of the second part the network is allowed. Plastic networks trained this way outperformed non-plastic recurrent networks on a pattern memorization task and competed with dedicated architectures on few-shot image classification, and a later extension, Backpropamine [32], replaced the fixed η by a neuromodulatory signal computed by the network itself. The price of differentiability is that the lifetime must be differentiable: the base weights and the rule are co-adapted by the same gradient, and the method does not apply when the lifetime is a closed-loop interaction with a physics simulator, which is why the works below, and this thesis, return to evolution.

Najarro and Risi [34] did exactly that, with the parametrization of (2.26) extended by a learning rate per synapse, so that every connection is described by five coefficients, and a natural evolution strategy [39] searching them. The weights of the network are redrawn uniformly at random at the start of every episode and organize themselves under the rules within a few dozen time steps, without any reward signal during the lifetime. On a simulated quadruped this produced the result that has since become the reference for the approach: a Hebbian network evolved on the intact body and on one damaged leg keeps walking when a leg it has never seen damaged is removed, where a network with evolved fixed weights falls. The same experiment also shows the cost: on the intact body the fixed-weight network walks farther than the plastic one. Plasticity buys robustness to a change of body at a price on the nominal body, and the random initialization at each episode is what forces the rules to do the whole job of building the controller.

Two later works ask where and how many rules are needed. Pedersen and Risi [37] evolve the rules of the same quadruped and periodically merge them by clustering, so that many synapses come to share one rule; the number of evolved coefficients drops by more than an order of magnitude and the smaller set generalizes better to unseen damage, a computational version of the genomic bottleneck. Ferigo et al. [13] evolve one rule per synapse in the controllers of voxel-based soft robots, whose body deforms as it moves, and report that the Hebbian controllers match or exceed static ones in locomotion and adapt to damage that was never seen during evolution.

Van Diggelen et al. [7] bring evolved Hebbian rules to swarm robotics. Each robot carries a small memoryless network with a rule of the form (2.26) on every synapse

and a fixed learning rate, and a single set of rule coefficients, evolved by CMA-ES (subsection 2.3.1) on the performance of the whole swarm, is shared by all its members. At the beginning of each trial every robot draws its own weights at random; since each robot then experiences a different sensory history, identical rules drive the weights of different robots to different places, and heterogeneous roles, including collective behavioural switching, emerge from a homogeneous genome. The rules outperform multi-agent reinforcement learning baselines on standard benchmarks and transfer from simulation to ten physical robots with a loss of under one percent. Two features of this design recur throughout the plasticity literature reviewed here: the controller converges from a random initialization at every trial, and the same rule set is used by every agent regardless of its individual situation.

2.5.3 Positioning of This Thesis

Read through (2.27), the co-design works above pay for the inner level in one of three ways:

1. *in full*, with a reinforcement learning run per body [17, 33];
2. *exactly solved*, but for an open-loop trajectory, which leaves the closed-loop controller out of the design [3];
3. *once*, with a policy that must be told which body it controls [18].

The plasticity works, for their part, all build the controller from random weights at every episode [14, 34, 13, 7] or co-train the base weights and the rule by the same gradient [31]; none places evolved rules on top of a policy that was pretrained by reinforcement learning and then frozen, and none uses plasticity across a population of bodies inside a co-design loop.

This thesis works in the intersection of the two gaps. Its controller is a policy pretrained by reinforcement learning and then frozen, made plastic only at its output layer by evolved rules of the form (2.26), so that the plastic part is, as in (2.28), a departure from a fixed part, except that the departure starts from trained weights rather than from random ones; and it is evaluated across a population of bodies inside a morphology search rather than on a single body. Why each of these choices is needed is the subject of chapter 3, and how they are realized of chapter 5.

Problems

The setting of this thesis is a winged drone flying through a forest of obstacles, judged on how far it gets and on the energy it spends getting there. Its body is described by a vector of morphological parameters and its controller is a neural network that reads the drone’s sensors and drives its actuators. For this chapter it is enough to keep in mind the nested problem of (2.27), in which a body m and a controller θ are judged together by a performance $F(m, \theta)$ that can only be measured by flying them.

The three sections that follow are three steps of one argument. The first shows what is lost when the body is fixed and only the controller is optimized; the second shows what is lost when the controller is fixed and only the body is optimized; the third shows that the obvious remedy, optimizing a controller for every body the search visits, cannot be paid for. What remains after the three steps is the requirement that the rest of the thesis sets out to meet: a controller that adapts to the body it is given, at a cost small enough to sit inside the body search.

3.1 The Sequential Design Problem

The usual way of building a flying robot is sequential. A body is designed first, by an engineer or by a commercial product, and a controller is then optimized for that body, whether by hand tuning, by trajectory optimization or, as in this thesis, by reinforcement learning. In the terms of (2.27) this fixes m at a chosen m_0 and solves only the inner problem, $\theta^*(m_0)$. What comes out is the best controller for that body, which is not the same thing as the best pair: any body m with $F(m, \theta^*(m)) > F(m_0, \theta^*(m_0))$ has simply never been tried.

That such bodies exist is not a technicality, because the body bounds what any controller can do. Energy per metre has a floor set by the airframe, its lift-to-drag ratio and the mass it carries, which no controller can change in flight. Manoeuvrability has a ceiling set by control authority, the forces and moments the actuators can produce, which no controller can exceed. A task that rewards both, distance through a cluttered forest and efficiency along the way, asks for a trade-off between the two, and a single hand-designed body sits at one point of that trade-off, chosen before the task was specified. Other points may be better on either objective, or dominate on both. Co-design studies report exactly this: evolved bodies outperform the conventional design they started from once each is given its own controller [17, 33, 3], and the conventional design was never the winner.

The sequential problem is therefore the reason to search over bodies at all: whatever the controller, a body that was never tried cannot be found.

3.2 The Morphology Optimization Problem

The mirror image of the sequential problem is to fix the controller and search over the body. This is the cheap way to co-design: a single controller θ_G , general enough to fly any body in the search space, scores every candidate, and the search maximizes $F(m, \theta_G)$ instead of $F(m, \theta^*(m))$. The two are not the same function of m , and the difference is not a matter of scale that a good optimizer could ignore. Two bodies can be ranked one way under the shared controller and the other way under their own:

$$F(m_1, \theta_G) > F(m_2, \theta_G) \quad \text{and} \quad F(m_1, \theta^*(m_1)) < F(m_2, \theta^*(m_2)), \quad (3.1)$$

in which case the search under θ_G keeps m_1 and discards the body that would have been better.

There is a systematic reason to expect (3.1) to hold often rather than rarely. A generalist controller is trained on a distribution of bodies and is, on any one of them, a compromise between all of them. The gap between what it achieves on a body and what a controller trained for that body alone achieves is smallest near the centre of the training distribution and grows towards its edges, where the body's dynamics differ most from the average the controller has learned to expect. But the edges are where the search wants to go: a body that is better than the reference is, almost by definition, one that departs from it, in size, in shape, in control authority, and the more it departs the more it is penalized by a controller that was never adapted to it. The fixed controller thus does two things at once. It undervalues exactly the bodies it should be finding, and it rewards the bodies it already flies well, which are the conservative ones. A search under a fixed generalist is biased towards the neighbourhood of its own training distribution, and no amount of evolutionary pressure removes a bias that is built into the fitness function.

The same argument applies one level up. Suppose the search under θ_G has returned a body, and a controller is then trained on that body alone. The pair obtained is the best controller for the best body under the generalist, which is not the best pair either: a body that ranked lower under θ_G but has more to gain from specialization, the m_2 of (3.1), is not recovered by training afterwards, because it was never kept. Specialization applied after the search cannot correct a search that did not account for it.

What the search needs, then, is not a controller that flies every body acceptably but a score that reflects what each body can do with a controller adapted to it. The next section shows why the direct way of obtaining such a score is closed.

3.3 The Computational Feasibility Problem

The remedy that the previous section seems to call for is the one adopted by the co-design works that train their controllers by reinforcement learning [17, 33]: solve the inner problem properly, by training a controller from scratch for every body the search evaluates. In the setting of this thesis the cost of doing so can be computed from two measured quantities, the training of a controller by reinforcement learning

and a search over bodies scored by rollouts alone, both executed on a node with two GPUs (NVIDIA V100 PCIe 32GB). Table 3.1 gives the figures.

Quantity	Value
Training of one generalist controller (1 000 PPO iterations, 65 536 parallel environments)	29 h
Bodies assessed by one body search (64 per generation, 50 generations)	3 200
One body search with one fixed controller for all bodies, rollouts only	94 h
One body search with a controller training per body, at the generalist’s cost	92 800 h
The same, with a specialist trained on the single body (about one hour each)	3 200 h

Table 3.1: Cost of a controller training inside the body search, from the production runs of this thesis. The last two rows are hypothetical; the others are measured.

A single training of the generalist takes just over a day, and a single body search visits 3200 bodies. Training a controller for each of them at that cost would take more than ten years on the same hardware. A specialist trained on one body alone is far cheaper, about an hour, and that is the realistic form of the remedy; even so, one training per body brings the search to more than four months, against the four days it takes when a body is scored by rollouts alone, a factor of more than thirty at the present size of the search. And the size is not fixed: every additional body, generation or scenario the search is given multiplies the gap again.

The multiplication is not optional. The performance of a body is a random quantity: forests are generated at random, the controller acts stochastically, and a single flight says little about a body. In this thesis each body is flown on sixty forests before it is ranked, and the ranking is still far from noise-free. Whatever the per-body cost is, it is paid that many times, and it is paid again every time the controller changes, because the score of a body under an old controller is worthless once the controller has moved. A co-design loop whose inner level is a training run therefore cannot scale on any of the axes the search needs: scenarios per body, bodies per generation and number of generations. The works that take this route stay under a thousand bodies in total [33] or require over a thousand processors for a single run [17], and both search far smaller design spaces, with far cheaper evaluations, than a winged drone in a forest.

The feasibility problem thus forbids the natural answer to the morphology problem and fixes the shape that any workable answer must have. The controller cannot be trained per body, so it must be paid for once, for the whole space of bodies. A controller paid for once is, by construction, not adapted to any one body, so its

adaptation to the body being scored must come from somewhere else and must cost no more than the rollouts that score the body anyway. Whether an adaptation that cheap can be enough to change which bodies the search finds is the empirical question of this thesis.

DA RILEGGERE Simulation Environment

4.1 Genesis: The Simulator

Every quantity optimized in this thesis is measured in simulation. A body is scored by flying it, a controller is trained by flying it, and the searches of the following chapters fly about a quarter of a million drones in every generation. The simulator is therefore the component that decides which experiments are affordable at all, and its internal organization leaves visible marks on the method. This work uses *Genesis* [15], an open-source physics engine for robotics released in December 2024, in its version 0.3.7. Genesis is written entirely in Python, but its numerical core is compiled at run time into code for the Graphics Processing Unit (GPU), so that tens of thousands of independent copies of the same scene advance in parallel on a single card. Genesis has no model of the air: the forces that keep a winged drone aloft are computed by a custom aerodynamic solver that is not part of the public release, was developed outside this thesis (it is credited in subsection 4.1.4) and runs inside the engine next to its rigid body solver.

This section describes how Genesis is organized and how its code reaches the GPU, weighs what that design gives and what it costs, and then follows one simulation step through its two halves: the rigid body dynamics that Genesis provides and the aerodynamic forces that were added to it. It closes with the random perturbations that are injected into both.

4.1.1 Architecture and Taichi Kernels

A Genesis program builds a *scene*. The scene owns a *simulator*, and the simulator owns a set of *solvers*, one for each physical model the engine supports: articulated rigid bodies, and a family of continuum models for liquids, granular media, cloth and deformable solids (the material point method, smoothed particle hydrodynamics, finite elements, position based dynamics and a grid based fluid solver). A *coupler* exchanges forces between solvers when objects of different kinds touch. The objects themselves are *entities*, each defined by a *morph* that gives its geometry (a primitive shape, a mesh, or a robot description file) and by a material that decides which solver is responsible for it. Only the solvers that own at least one entity are stepped. In this thesis the scene contains a single entity, the drone, loaded from a Unified Robot Description Format (URDF) file that is generated from the genome of the body; it is handled by the rigid body solver, and the aerodynamic solver acts on it as one more solver of the same simulator. Rendering is a separate subsystem and stays switched off during training and search.

None of the numerical work is done by the Python interpreter. Genesis is built on *Taichi* [24, 25], a domain specific language for parallel numerical computation that is embedded in Python, and of which the Genesis developers maintain their own fork.

A Taichi *kernel* is a function written in a statically typed subset of Python syntax and marked with a decorator. The first time a kernel is called, Taichi does not execute its body: it reads the syntax tree of the function, translates it into its own intermediate representation, applies the usual optimizations of a compiler (constant folding, elimination of common subexpressions and of dead branches, simplification of memory accesses) and lowers the result through LLVM to machine code for the selected backend, which here is CUDA. This is *just-in-time* compilation: the program is compiled while it runs, the first time each piece is needed, and every later call launches the compiled binary directly (Figure 4.1).

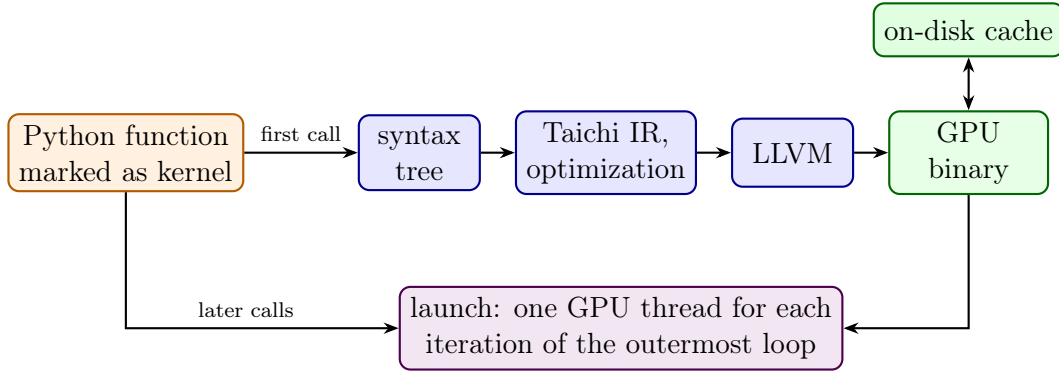


Figure 4.1: Life of a Taichi kernel. The first call compiles the Python function down to a GPU binary, which is also written to a cache on disk; later calls, and later runs of the same program, skip the compilation and only launch the binary. IR stands for intermediate representation.

What makes a kernel parallel is a single rule: the outermost `for` loop of a kernel is parallelized automatically, and on a GPU every iteration of that loop becomes one thread. The body of the loop can be arbitrary scalar code, with branches, local variables, inner loops and calls to other Taichi functions, and all of it is fused into one GPU program. This is the opposite of the way a tensor library such as PyTorch reaches the GPU, where every elementary operation is a separate GPU program that reads and writes whole arrays. Physics code is full of logic that differs from element to element (a joint limit that is active in one copy of the scene and not in the next, a wing that is stalled here and not there), and it is awkward and wasteful to express as a chain of whole-array operations, while inside a kernel it is an ordinary `if`. Every kernel launch from Python has a small fixed cost, so the engine is organized as a few large kernels rather than many small ones.

The data that kernels work on live in *fields*: typed, multidimensional arrays that are allocated on the GPU and declared separately from the kernels that use them. Keeping the description of the data apart from the computation is the central idea of Taichi [24], and it has a consequence that matters in practice. A kernel refers to its fields as global variables, so at compilation time it is bound to those specific fields and to their shapes; and the parameters that are declared static (the options of a solver, the number of links of a robot, which branches of the code are enabled) are compiled in as constants, with the unused branches removed. A compiled kernel is therefore specialized to one scene. Genesis makes this explicit by separating the construction of a scene from its *build*: entities and options are declared first, and a

single build call then allocates every field and fixes every static parameter. After the build, the values stored in the fields can change freely, which covers the state of the simulation and the physical parameters that were declared as varying, but nothing structural can: not the number of bodies, not their topology or geometry, not the size of any array. A structural change requires a new scene and a new build.

The most important structural parameter is the number of parallel *environments* B , given to the build call. The B environments are independent copies of the scene that share one model (the kinematic tree, the geometry, the nominal inertias) and differ in their state. Every state field has an axis of length B : the generalized positions form an array of size $n_q \times B$, the joint space inertia matrix an array of size $n_v \times n_v \times B$, and the parallel loop of each kernel runs over pairs (element, environment), so that a single launch advances every link of every environment. Environments never interact, which is exactly the structure of the workloads of this thesis: many flights of the same body, each in its own forest. Finally, the fields can be read and written as PyTorch tensors that reside on the same GPU: the exchange is a copy between two regions of the memory of the card and never passes through the host. The neural controllers, the obstacle fields and the reward computations are PyTorch code, the physics is Taichi code, and during a run only logging scalars ever leave the GPU.

4.1.2 Advantages and Disadvantages

The first reason to choose Genesis is throughput. The generalist controllers of this thesis are trained on 65 536 simultaneous flights and a single generation of the body search scores 245 760 of them; with the whole loop of physics, perception and control resident on the GPU, these sizes are reachable on one or two cards, and the costs reported in Table 3.1 are the measured result. The second reason is that the engine is open to extension at the level where it matters. Because a solver is a Python class whose numerical methods are Taichi kernels, a new physical model can be written in the same language as the engine, registered next to the built-in solvers, and compiled into the same GPU pipeline with direct access to the state of the rigid bodies. The aerodynamic solver used in this thesis is such an extension, and it evaluates the forces on every surface of every drone in one kernel launch per step. The same holds for perception: the depth sensor of the drone is a custom kernel that intersects rays with the trees of each environment. The third reason is the interface. Bodies are read from standard robot description files, which a script can generate from a genome; the state is exposed as batched PyTorch tensors, which is the format that reinforcement learning libraries expect; and physical parameters such as the masses and the centres of mass of the links can be perturbed independently in every environment. Lastly, Genesis is open source, so the whole stack can be packaged in a container and run unchanged on a workstation and on a cluster.

The costs follow from the same design. Compilation is slow: the first build of a scene compiles hundreds of kernels and can take several minutes, against the milliseconds of a simulation step. The cache on disk removes most of this cost from later runs, but a build still has to allocate and initialize the scene, and it is paid for every scene. This matters for co-design more than for ordinary robot learning,

because all the environments of a scene share one body: a population of different bodies needs one scene per body, and replacing the bodies means building new scenes. Two further restrictions make this worse. Genesis can be initialized only once in a process and all its scenes share one Taichi runtime, so the scenes of a process are stepped one after the other; real parallelism across scenes is obtained with separate processes, each with its own share of the GPU memory. And since kernels are bound to their fields when they are compiled, a program that replaces a field after the first launch (for instance to change the number of trees) leaves the compiled kernel reading the old one, silently. Every array whose size might change has to be dimensioned for the worst case at build time, or its owner rebuilt.

Numerical precision is a second limitation. Genesis runs in 32-bit floating point by default, with the relaxed arithmetic rules that compilers use for speed, and the order in which parallel GPU threads accumulate sums is not guaranteed. Two runs with the same random seed do not produce the same trajectories: the differences start at the last bit and, because a flight among obstacles is a chaotic system, grow until the two flights end differently. The outcome of a flight must therefore be treated as a random variable even when every explicit source of randomness is held fixed, and a fitness value is only meaningful as an average over many flights.

Finally, Genesis is a young project. Its programming interface still changes between releases and parts of its documentation lag behind the code. The headline figure published with its release, 43 million simulation steps per second for a robot arm on a single consumer GPU, was obtained in conditions (a motionless robot, self-collisions disabled) that an independent evaluation found unrepresentative of robot learning, where it measured a throughput lower by two orders of magnitude [48]; the only figures relied upon in this thesis are those measured on its own workloads. For fixed-wing flight it offers nothing out of the box: its only aerial model is a quadrotor whose propellers produce a thrust proportional to the square of their rotation speed. Everything aerodynamic in this thesis comes from a solver added to the engine, and because an added solver exchanges forces with the rigid body solver explicitly, once per substep, its numerical stability has to be secured by a small time step and by limits on the forces, as described below.

4.1.3 Physics Simulation

The drone is an articulated rigid body: a fuselage that moves freely in space and carries six revolute joints, driven by servomotors, which sweep and twist the two halves of the wing and deflect the elevator and the rudder (the body is described in section 4.3). Its configuration q collects the position of the fuselage, its orientation as a unit quaternion and the six joint angles, for $n_q = 13$ numbers, and its generalized velocity v collects the linear and angular velocity of the fuselage and the six joint rates, for $n_v = 12$ degrees of freedom. The rigid body solver of Genesis follows the formulation of MuJoCo [51], and advances the equation of motion in generalized coordinates,

$$M(q) \dot{v} + c(q, v) = \tau_{\text{act}} + \tau_{\text{pas}} + J_a(q)^\top F_a + J_c(q)^\top f_c, \quad (4.1)$$

where M is the joint space inertia matrix, c gathers the Coriolis, centrifugal and gravitational terms, τ_{act} and τ_{pas} are the actuator and the passive joint torques, F_a

stacks the aerodynamic forces with J_a the Jacobian of their points of application, and f_c are the constraint forces with Jacobian J_c . At every substep the solver computes M with the composite rigid body algorithm and c with the recursive Newton-Euler algorithm [12], both of which traverse the kinematic tree in a time linear in the number of links, and factorizes M once for all the linear systems that follow.

The servomotors are modelled as proportional-derivative position controllers with a bounded torque,

$$\tau_{\text{act},j} = \text{clip}(k_p (q_j^* - q_j) - k_v v_j, -\tau_j^{\max}, \tau_j^{\max}), \quad (4.2)$$

with $k_p = 8 \text{ N m/rad}$, $k_v = 2 \text{ N m s/rad}$ and a torque limit of 1.5 N m for the wing joints and 1 N m for the tail joints. The target angle q_j^* is what the controller commands. The passive torques are a viscous damping and a dry friction in each joint, and an armature term, which stands for the reflected inertia of the motor and of its gears, is added to the diagonal of M . The response of a control surface to a command is thus not instantaneous: it is the response of a damped second order system with a saturating motor, and its delay is part of what the controller has to cope with.

Constraints are handled as in MuJoCo, by a convex optimization over accelerations rather than by a complementarity problem [50]. Let $a_0 = M^{-1}(\tau_{\text{act}} + \tau_{\text{pas}} + J_a^\top F_a - c)$ be the acceleration the system would have without constraints. The solver selects

$$\begin{aligned} \dot{v} = \arg \min_a \frac{1}{2} (a - a_0)^\top M (a - a_0) + \sum_{i \in \mathcal{A}(a)} \frac{1}{2} d_i (J_i a - a_i^{\text{ref}})^2, \\ a_i^{\text{ref}} = -b_i J_i v - k_i r_i, \end{aligned} \quad (4.3)$$

that is, the acceleration closest to the unconstrained one in the metric of the kinetic energy (Gauss's principle of least constraint), softened by a penalty on each active constraint i . The reference acceleration a_i^{ref} is that of a critically damped spring which would bring the violation r_i back to zero within a chosen time constant, the weight d_i sets how stiffly the constraint is enforced, and the set $\mathcal{A}(a)$ contains the inequality constraints that are currently pushing in their admissible direction. The problem is convex and has a unique solution, and the constraint forces f_c follow from it. Genesis offers a Newton and a conjugate gradient method for (4.3); this thesis uses the latter. In the scenes of this work the problem is small, because contacts are disabled (see below): the only constraints are the mechanical limits of the six joints, which become active only when a joint reaches its stop, and the dry friction of the joints.

Time integration is semi-implicit Euler. With substep Δt ,

$$v_{k+1} = v_k + \Delta t \dot{v}_k, \quad q_{k+1} = q_k \oplus \Delta t v_{k+1}, \quad (4.4)$$

where the new velocity is used to advance the positions and $\oplus$ denotes ordinary addition for positions and joint angles and the composition with the rotation of axis and angle $\Delta t \omega_{k+1}$ for the quaternion, which thereby stays of unit norm. The default integrator of Genesis, used here, makes one modification for stiff actuators: the joint damping D and the derivative gains K_v of the servomotors are treated implicitly,

which amounts to replacing M with $M + \Delta t (D + K_v)$ in the computation of the acceleration and keeps the servo loop stable whatever its gains. All other forces, the aerodynamic ones included, are explicit: they are evaluated on the state at the beginning of the substep and held constant across it.

Two time scales are used. The controller acts at 25 Hz, and every control step of 0.04 s is divided into four physics substeps of $\Delta t = 0.01$ s, so that the dynamics and the aerodynamic forces are updated at 100 Hz while commands are held (Figure 4.2). The finer physics step is what the explicit aerodynamic coupling requires: a lifting surface reacts to a change of incidence with a force that opposes it, which makes the equations stiff, and an explicit scheme remains stable only if the step is small compared with the fastest of these reactions.

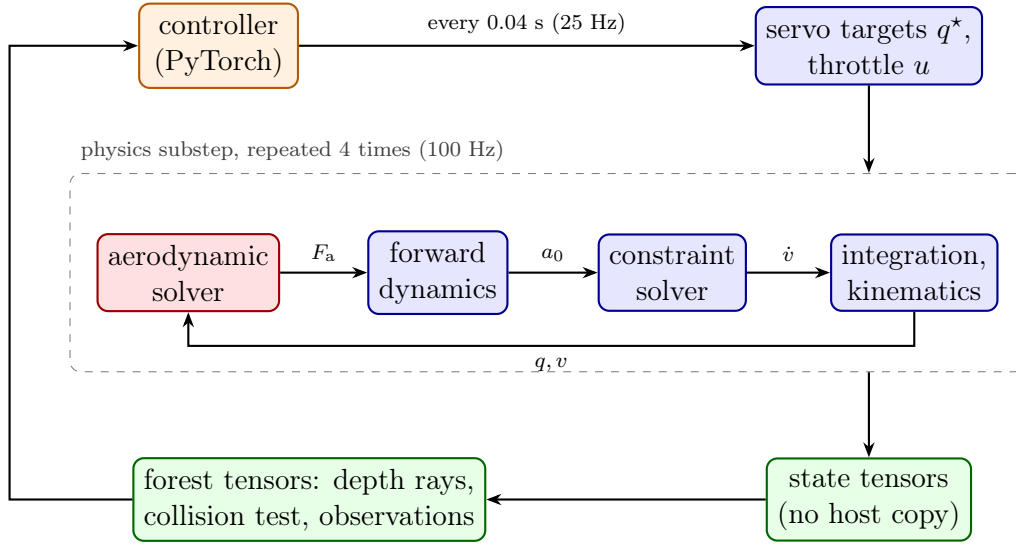


Figure 4.2: One control step. The commands of the controller are held while four physics substeps are executed; in each of them the aerodynamic solver turns the current state into forces, and the rigid body solver of Genesis computes the unconstrained acceleration, enforces the joint limits and integrates. Everything in the figure runs on the GPU, for all environments at once.

Genesis has a complete collision pipeline: a broad phase that sorts axis-aligned bounding boxes to discard distant pairs, a narrow phase for convex shapes, a convex decomposition for meshes that are not convex, and contact forces with Coulomb friction that enter (4.3) as further constraints. None of it is used here, and the trees of the forest are not objects of the physics engine at all. There are two reasons. A flight ends the moment the drone touches anything, so no contact ever has to be resolved, only detected. And the geometry of a scene is shared by all of its environments, whereas this work needs a different random forest in every environment. The forests are therefore stored outside the engine, as tensors that hold the position of every trunk in every environment, and the two operations that involve them, the depth sensing of the drone and the detection of a collision, are computed geometrically on those tensors for all environments at once (section 4.2). Likewise there is no ground plane: a flight that descends below a minimum altitude is ended.

4.1.4 Aerodynamics Simulation

The aerodynamic solver described in this subsection is not a contribution of this thesis. It was designed and implemented by Andrea Vicari as part of his doctoral research, which is still unpublished at the time of writing, and it is used here as he provided it, as the aerodynamic model of every simulation in this work. The credit for this part of the simulator is entirely his, and the author is grateful to him for making it available. What follows documents the model as it was used, so that the results of this thesis can be read and reproduced; it should not be taken as the reference description of his work, which will be his own publication.

The aerodynamic solver computes, at every physics substep and for every environment, the force that the air exerts on each part of the drone, and hands these forces to the rigid body solver as the term $J_a^\top F_a$ of (4.1). It is a *quasi-steady component build-up*: the aircraft is divided into a small number of elements, each element receives one lumped force that depends only on the instantaneous flow it sees, through lift and drag coefficients of closed form, and the forces are summed by the rigid body dynamics. A model of this kind is orders of magnitude cheaper than any solution of the flow field, which is what allows it to run a hundred times per simulated second in a quarter of a million environments. It is also algebraic in the geometry of the body, so that a new body needs no aerodynamic data of its own: areas, chords, spans and aspect ratios are read from the same URDF file that defines the rigid body, and the coefficients follow from them.

Elements and local flow. The drone has seven aerodynamic elements: the fuselage, the two halves of the wing, the two halves of the elevator, the rudder and the propeller. Each lifting surface carries a reference frame at the quarter of its chord, with the x axis along the chord towards the trailing edge, the y axis along the span and the z axis normal to the surface. The frame is rigidly attached to the link of the surface and moves with it, and this is how the morphing of the wing enters the aerodynamics without any dedicated model: a twist of a wing half pitches its frame and changes its angle of attack, a sweep yaws the frame, changes its sideslip and displaces its point of application. The air is assumed at rest. If v_{cm} is the velocity of the centre of mass of the drone and R_i the orientation of the frame of element i , the relative wind seen by the element and its aerodynamic angles are

$$\begin{aligned} w_i &= -R_i^\top v_{\text{cm}}, & V_i &= \|w_i\|, \\ \alpha_i &= \text{atan2}(w_{i,z}, w_{i,x}), & \beta_i &= \arcsin \frac{w_{i,y}}{V_i}, \end{aligned} \tag{4.5}$$

the angle of attack α_i and the sideslip angle β_i . The index i is dropped in what follows.

Lift and drag coefficients. For small angles of attack a lifting surface behaves according to the classical theory of the finite wing [1]. The slope of its lift curve is lower than that of its airfoil section, C_{ℓ_α} , because the vortices shed at the tips reduce the effective incidence, and the reduction grows as the aspect ratio AR of the surface decreases. The solver uses the formula of Helmbold, which unlike the better

known lifting-line result remains accurate for the low aspect ratios of a small drone:

$$C_{L\alpha} = C_{\ell\alpha} \frac{AR}{2 + \sqrt{AR^2 + 4}}. \quad (4.6)$$

Lift is linear in the angle of attack measured from the zero-lift angle α_0 of the section, and drag is the sum of a parasitic part and of the drag induced by lift:

$$C_L^{\text{lin}} = f_{Re} C_{L\alpha} (\alpha - \alpha_0), \quad C_D^{\text{lin}} = C_{D_0} + \frac{(C_L^{\text{lin}})^2}{\pi e AR}, \quad (4.7)$$

with $e = 0.8$ the Oswald efficiency factor. The factor $f_{Re} = \min(Re/Re_{\text{nom}}, 1)$, where $Re = \rho V \bar{c}/\mu$ is the Reynolds number of the surface computed on its mean chord $\bar{c}$, reduces the lift of a surface that operates below the nominal Reynolds number of its section, as happens to a narrow wing in slow flight. The aspect ratio in (4.6) and (4.7) is that of the complete wing, including the part of the span covered by the fuselage, with the span reduced by the factor $\cos \beta$ when the surface is swept or the drone flies with sideslip.

The linear regime ends at stall, and a controller that manoeuvres among obstacles, or a body that is badly proportioned, will cross it; a model that stops at stall would make every such event a numerical accident. Beyond stall the solver therefore switches to the coefficients of a flat plate in fully separated flow,

$$C_L^{\text{fp}} = f_{Re} \sin 2\alpha, \quad C_D^{\text{fp}} = 2 f_{Re} \sin^2 \alpha, \quad (4.8)$$

which are defined for any incidence, reversed flow included, and give the correct limits: no lift and the drag of a plate normal to the wind at $\alpha = 90^\circ$. The two regimes are joined by a smooth switch centred on the stall angle α_s , as is common in the flight dynamics models of small unmanned aircraft [2]:

$$\sigma(\alpha) = \frac{1}{2} \left[1 + \tanh \frac{|\alpha| - \alpha_s}{m \alpha_s} \right], \quad (4.9)$$

$$C_L = (1 - \sigma) C_L^{\text{lin}} + \sigma C_L^{\text{fp}}, \quad C_D = (1 - \sigma) C_D^{\text{lin}} + \sigma C_D^{\text{fp}},$$

with $m = 0.2$ the relative width of the transition. A finite wing stalls at a larger angle than its section, because it needs more incidence to reach the same lift. The solver accounts for this by computing the stall angle of the surface from that of the section, α_s^{2D} :

$$\alpha_s = \alpha_0 + \frac{2\pi \delta}{C_{L\alpha} (1 + 2\delta/AR)}, \quad \delta = \alpha_s^{2D} - \alpha_0. \quad (4.10)$$

The numerator is the maximum lift coefficient that thin airfoil theory assigns to the section, and the division by $C_{L\alpha}$ gives the incidence at which the finite surface reaches it; the term in brackets is an empirical correction that lowers the result for short wings. The correction is applied to the wing and to the elevator, while the rudder keeps the stall angle of its section. Figure 4.3 shows the resulting curves for the wing of the reference drone.

The parameters of the section, α_0 , α_s^{2D} , C_{D_0} and Re_{nom} , are not free constants. The wing section is a four-digit NACA airfoil chosen by the genome of the body,

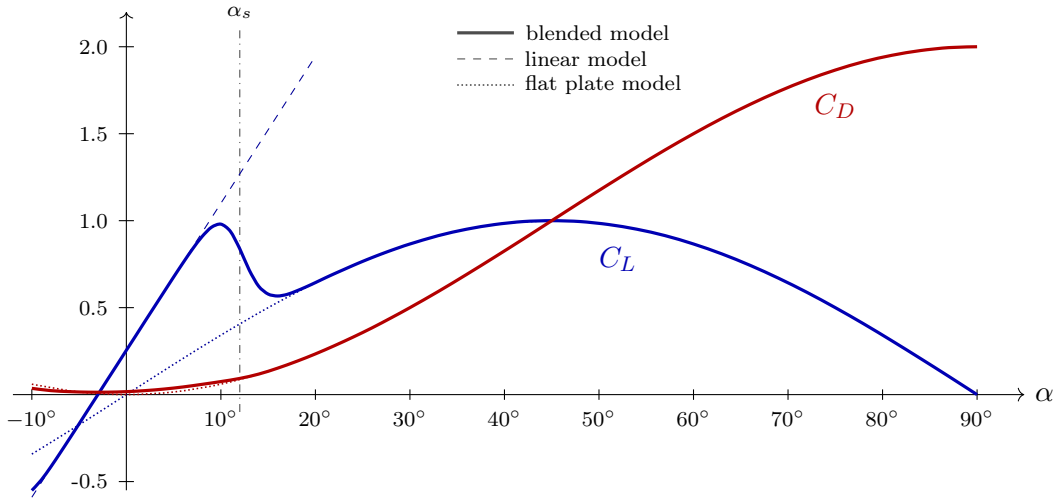


Figure 4.3: Lift and drag coefficients of the aerodynamic model for the wing of the reference drone (NACA 3416 section, aspect ratio 7.5). The dashed line is the linear model (4.7) (shown for the lift only), the dotted lines are the flat plate model (4.8), and the solid lines are their blend (4.9) around the stall angle $\alpha_s = 12.0^\circ$ of the finite wing. The maximum lift coefficient is 0.98 and the best lift to drag ratio of the isolated wing is 18.7.

and its parameters are read from a table that was computed in advance, for every admissible airfoil, from polars obtained with the panel code XFOIL [8] over a sweep of Reynolds numbers. The slope of the section is fixed at $C_{\ell_\alpha} = 0.11$ per degree, the value that thin airfoil theory predicts to within one percent. The elevator and the rudder always use the same symmetric section. The fuselage produces no lift and a constant drag coefficient of 0.35 referred to its own reference area.

Forces and points of application. With the dynamic pressure $\frac{1}{2}\rho V^2$, the air density $\rho = 1.225 \text{ kg/m}^3$ and the area S of the element, lift and drag are

$$L = \frac{1}{2} \rho V^2 S C_L \cos^2 \beta, \quad D = \frac{1}{2} \rho V^2 S C_D \cos^2 \beta, \quad (4.11)$$

where the factor $\cos^2 \beta$, applied to the wing and to the elevator, expresses the classical result that only the component of the flow normal to the span of a surface loads it. The two magnitudes are assembled in wind axes and rotated into the frame of the surface through the two aerodynamic angles,

$$F = R_z(\beta) R_y(\alpha) (D, 0, L)^\top, \quad (4.12)$$

with R_y and R_z the elementary rotations about the y and z axes of the frame. The rudder is treated as a wing turned by ninety degrees: its incidence is the sideslip angle, its coefficients are evaluated at β , and its lift acts as a side force along y . The force is not applied at the origin of the frame but at the *centre of pressure* of the surface, which for an attached flow lies at the quarter of the chord and moves towards the middle of the chord as the flow separates:

$$\frac{x_{cp}}{\bar{c}} = \frac{1}{4} + \frac{1}{4} \frac{|\alpha|}{\pi/2}. \quad (4.13)$$

Since the rigid body solver receives a force and a point, the aerodynamic moments of the drone are never modelled as such: the pitching moment that trims the aircraft, the rolling moment of a differential twist and the yawing moment of an asymmetric sweep all arise as moments of the forces (4.12) about the centre of mass, and they change with the body because the points of application do.

Interaction between the elements. Two interactions are modelled, both directed at the tail. The first is the *downwash* of the wing: a wing that produces lift deflects the flow behind it downwards, and the elevator works in that deflected flow. Its angle of attack is reduced by the downwash angle that lifting-line theory predicts at the wing,

$$\alpha_t = \alpha - \varepsilon, \quad \varepsilon = k_\varepsilon \frac{C_{L,w}}{\pi AR_w}, \quad (4.14)$$

where $C_{L,w}$ is the lift coefficient that the wing half on the same side has at that same substep, AR_w the aspect ratio of the wing and $k_\varepsilon = 1$. This couples the two surfaces the way it does on a real aircraft (an increase of wing lift reduces the effectiveness of the tail) and makes the two halves of the elevator work differently when the two halves of the wing do. For this reason the kernel evaluates the elements in a fixed order: the propeller first, then the wing and the fuselage, then the tail. The second interaction is the *slipstream* of the propeller, whose induced velocity, given below, corrects the axial component of the flow seen by the elevator through a gain k_s .

Propeller. The throttle command $u \in [0, 1]$ does not set the thrust directly. It passes through a first order lag that represents the inertia of the motor and of the propeller,

$$\tilde{u}_{k+1} = \tilde{u}_k + \frac{\Delta t}{\Delta t + 1/(2\pi f_c)} (u_k - \tilde{u}_k), \quad (4.15)$$

with a cut-off frequency $f_c = 0.32$ Hz, and the filtered command sets the rotation speed $n = \tilde{u} n_{\max}$ (in revolutions per second), where $n_{\max}$ is the smaller of the no-load speed of the motor, $K_V U/60$ for a speed constant $K_V = 2300$ rpm/V and a supply voltage $U = 7.4$ V, and of the speed at which the propeller delivers its maximum static thrust $T_{\max} = 5.2$ N. The thrust of a propeller of diameter d falls as the aircraft flies faster, because the blades meet the air at a lower incidence; this is described by a thrust coefficient that decreases with the *advance ratio* J [29]:

$$J = \frac{V_a}{n d}, \quad T = \rho n^2 d^4 C_{T_0} \max(0, 1 - c_1 J - c_2 J^2), \quad (4.16)$$

with V_a the component of the relative wind along the axis of the propeller, $C_{T_0} = 0.093$, $c_1 = 0$ and $c_2 = 2.148$, and T limited to the interval $[0, T_{\max}]$. The thrust acts along the axis, together with a reaction torque κT about it, $\kappa = 0.01$ m. The velocity induced in the slipstream follows from the momentum theory of the actuator disc of area $A = \pi d^2/4$:

$$v_{\text{ind}} = \frac{1}{2} \left(-V_a + \sqrt{V_a^2 + \frac{2T}{\rho A}} \right). \quad (4.17)$$

The dependence of the thrust on the flight speed matters for the two objectives of this thesis. A model with a thrust proportional to the throttle would make fast flight as cheap as slow flight; with (4.16) the propeller loses effectiveness with speed, and the energy spent per metre depends on how well the drag of the body matches the propeller over the range of speeds at which the controller chooses to fly.

Coupling and safeguards. At each substep the forces of all elements, with their points of application, are transformed to the world frame and accumulated on the links of the rigid body as forces and moments. Three safeguards protect the explicit coupling. The speed V that enters the coefficients is limited to 40 m/s, the force on each element is limited to 30 N, several times the weight of the drone, and values that are not finite are replaced by zero before they reach the rigid body solver. In a parallel simulation this last point is not cosmetic: the environments of a scene are advanced by the same kernels on shared arrays, and a single environment whose state diverges can otherwise spread its invalid values to the rest of the batch.

Limits of the model. The model is a compromise between fidelity and the cost of evaluating it a hundred times per second in every environment, and its simplifications should be stated. It is quasi-steady: forces depend on the instantaneous flow only, so unsteady effects such as dynamic stall or the lag of the downwash in reaching the tail are absent. Each surface carries a single lumped force, so the distribution of the load along the span, and with it the progressive stall of a twisted or swept wing, is not resolved. The relative wind (4.5) of every element is computed from the velocity of the centre of mass: the additional flow that a rotation of the body induces at a surface far from the centre of mass is neglected, and the rotations of the drone are damped only indirectly, through the change of incidence they produce. Interference is limited to the two effects described above. Finally, the air is at rest: there is no wind and no turbulence other than the noise described in the next subsection. None of this prevents the model from ranking bodies by the quantities that drive the search (how much wing a body has, how it is proportioned, where its surfaces sit with respect to its centre of mass, which section it uses), but the absolute values of range and energy it produces are those of the model and would need to be validated against flight data before being read as predictions.

4.1.5 Simulation Noise

A deterministic simulator invites overfitting. A controller trained by reinforcement learning, and even more an evolutionary search that compares bodies by small differences in fitness, will exploit any regularity of the simulator that helps, including those that have no counterpart in reality: a lift curve known to the last digit, a centre of pressure that never wanders, a mass distribution that is exactly the nominal one. The standard countermeasure is to randomize the simulator, so that a solution can only score well if it works across a family of slightly different worlds [49, 38]. In this thesis noise is injected at three levels of the simulation.

The first level is the aerodynamic force itself. At every physics substep, and independently for every lifting surface of every environment, the force (4.12) is perturbed

in magnitude and in direction before it is applied:

$$\begin{aligned} \tilde{F} &= \|F\| (1 + \sigma_m s \xi) \frac{\hat{F} + \sigma_d s g_\perp}{\|\hat{F} + \sigma_d s g_\perp\|}, \\ s(\alpha, \beta) &= 1 + 3 \left(\frac{\min(\sqrt{\alpha^2 + \beta^2}, \pi)}{\pi} \right)^2, \end{aligned} \quad (4.18)$$

where $\hat{F}$ is the unit vector of F , ξ a standard normal variable, $g_\perp$ a random unit vector orthogonal to F , and $\sigma_m = \sigma_d = 0.05$. The scale s grows from 1 in aligned flow to 4 in fully reversed flow, so that the force is least reliable exactly where the model is: at large incidence, around and beyond stall. The centre of pressure is displaced at the same time by a random offset whose standard deviation is a fraction $\sigma_{cp} s/4$ of the chord, with $\sigma_{cp} = 0.01$, limited to 15% of the chord. The thrust of the propeller is not perturbed. Being drawn anew a hundred times per second, this noise acts on the drone as a broadband buffeting: its effect on the trajectory partly averages out, but it denies the controller a perfectly repeatable response to its commands.

The second level is the body. At the beginning of every flight the mass of each link is multiplied by a factor drawn from a normal distribution with mean one and standard deviation 2%, and its centre of mass is displaced along each axis by a normal offset of standard deviation 4 mm. Both remain fixed for the duration of the flight. Genesis supports these perturbations natively: although the environments of a scene share one model, the mass and the centre of mass of every link carry an offset that is specific to the environment. For a winged drone the position of the centre of mass relative to the wing decides the stability of the aircraft, so a few millimetres are a meaningful disturbance, of the size that a real airframe shows from one assembly to the next.

The third level is the interface between the simulator and the controller: the observations are corrupted by sensor noise, the commands reach the servomotors with a random latency of up to one control step and with a small random offset, and the forest and the initial conditions are drawn at random for every flight. These belong to the definition of the task rather than to the simulator and are described with it.

The consequence is that the outcome of a flight, and therefore every quantity derived from it, is a random variable, by design and, as noted above, also because of the arithmetic of the GPU. A single flight says little about a body or about a controller. Throughout this thesis performance is an average over many flights in different forests, and the comparisons between bodies and between methods are made with that variability in view.

4.2 Flight Environment

The task is the same in every experiment of this thesis: fly forward through a forest, as far as possible. The forest stands in a straight corridor. With x the direction of flight, y the lateral direction and z the vertical, the corridor is $W = 100$ m wide ($|y| \leq 50$ m), the trees occupy the stretch $0 \leq x \leq L$, and the drone is released

30 m before the first tree, on the axis of the corridor, 15 m above the ground and heading along x . A tree is a vertical cylinder of radius $r = 0.75$ m and is treated as infinitely tall, so that a forest cannot be flown over: avoiding the trees is a planar problem, and the altitude matters only to the flight itself.

As anticipated in subsection 4.1.3, the trees are not objects of the physics engine. A forest is nothing more than the list of the positions of its trunks, and a batch of F forests of T trees each is a single tensor of $F \times T$ positions that resides on the GPU. At the beginning of every flight an environment is assigned one of the F forests, and from then on the forest enters the simulation through two operations only, which are computed for all environments at once: what the drone sees, which is described with its sensors in subsection 4.3.2, and whether it has hit something.

Collision. Let (x_b, y_b) be the position of the centre of a trunk in the horizontal frame centred on the drone and aligned with its heading, b the wingspan of the body (the sum of its two wing halves) and ϕ its roll angle. The drone has hit the trunk when

$$|x_b| \leq r + \tau \quad \text{and} \quad |y_b| \leq \frac{b}{2} \cos \phi + r + \tau, \quad (4.19)$$

that is, when the trunk overlaps the segment that the wing projects on the ground, which shortens as the drone banks (the rectangle is drawn in Figure 4.9). A drone can therefore pass through a gap narrower than its wingspan, but only by rolling steeply while it does so. The margin τ is 1 cm when a body is evaluated and 20 cm during the training of the controllers, which teaches them to keep some distance from the trunks.

End of a flight. A flight ends when the drone hits a trunk, leaves the corridor sideways, descends to the ground, exceeds a quarter of a turn in roll, pitch or yaw (it has tumbled, or turned back), flies past the end of the forest, or reaches a time limit. The coordinate x at which the flight ends is its *progress*, the first of the two quantities by which bodies and controllers are judged in this thesis; the second is the energy spent per metre.

What remains to be defined is how the positions of the trunks are drawn, and the choice is less innocent than it seems, because the forest decides what the progress of a flight measures. Three generators were written in the course of this work. They share the corridor and the trees described above and differ only in the law of the positions: one was used for internal tests, one produced every result of this thesis, and one was explored and then abandoned. The three are shown below in the same corridor and with nearly the same number of trees, so that the figures can be compared directly. The lengths and densities used in the experiments are given with the method.

4.2.1 Uniform Density Forests

The simplest forest places a fixed number N of trees independently and uniformly in the rectangle $[0, L] \times [-W/2, W/2]$ (Figure 4.4). The density is the same everywhere, $N/(LW)$ trees per square metre, or, in the unit used throughout this section, $\lambda = N/L$ trees per metre of course.

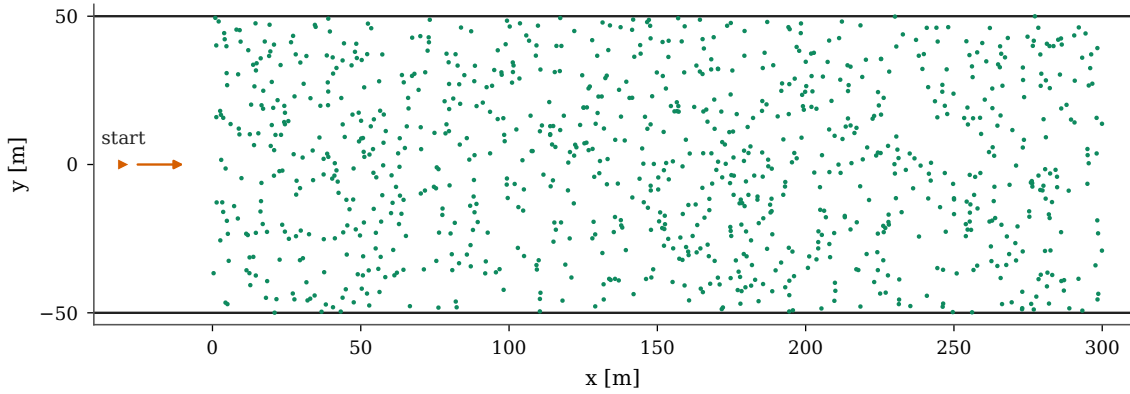


Figure 4.4: A uniform density forest seen from above: 975 trees placed independently and uniformly in a corridor of 300 m by 100 m. Trunks are drawn to scale. The thickets and the clearings are not designed: they are what independent positions look like.

A uniform forest is statistically the same at every point of the course, and this is what makes it useful for testing. It was used to verify the depth sensor and the collision test, to debug the flight of new bodies, and to observe how a controller behaves at one given density, without the confounding effect of a forest that changes as the drone advances.

The same property makes it a poor instrument for measuring. The difficulty of a uniform forest is a single number that has to be chosen in advance, and no choice suits a population of bodies of very different ability: in a sparse forest every competent body reaches the end and the progress saturates, in a dense one they all fail within a few metres. Between these extremes the measure is dominated by chance. The cleanest way to see it is to consider a drone that flies straight and blind. It touches a trunk whenever a centre falls in the strip of width $w = b + 2r$ swept by its wing, and since the positions are independent, the probability that it is still flying at x is

$$S(x) = \exp\left(-\frac{w}{W} \int_0^x \lambda(\xi) d\xi\right), \quad (4.20)$$

which for a constant λ is an exponential law: the risk of the next metre is the same whatever the distance already flown, and the distance covered has a standard deviation equal to its mean. In the forest of Figure 4.4, with the wingspan $b = 1.4$ m of the reference drone, that mean is 11 m. A controller that sees the trees does much better than this, but it inherits the structure of the problem: in a stationary forest the length of a flight has a large memoryless component, and many flights are needed before two bodies can be told apart.

4.2.2 Growing Density Forests

In the forests used for all the results of this thesis the density grows linearly along the course, from λ_0 trees per metre at the entrance to λ_1 at the end:

$$\lambda(x) = \lambda_0 + (\lambda_1 - \lambda_0) \frac{x}{L}. \quad (4.21)$$

The number of trees is the integral of the profile, $N = \lceil (\lambda_0 + \lambda_1)L/2 \rceil$. The lateral coordinate of each tree is uniform across the corridor, as before, and its coordinate along the course is drawn from the density proportional to (4.21) by inverting its cumulative distribution, which for a linear profile has a closed form: if u is uniform in $[0, 1]$,

$$x = L \frac{\sqrt{\lambda_0^2 + u(\lambda_1^2 - \lambda_0^2)} - \lambda_0}{\lambda_1 - \lambda_0}. \quad (4.22)$$

A whole batch of forests is thus generated with a handful of tensor operations. The entrance density λ_0 can also be drawn at random for each forest within a given range, an option used during training so that the controller meets dense forest at the beginning of a flight as well as at the end. Since the forests of a batch share one tensor and may have different numbers of trees, the shorter lists are padded with inactive trees parked outside the corridor, where they can be neither seen nor hit. Figure 4.5 shows one forest and its profile.

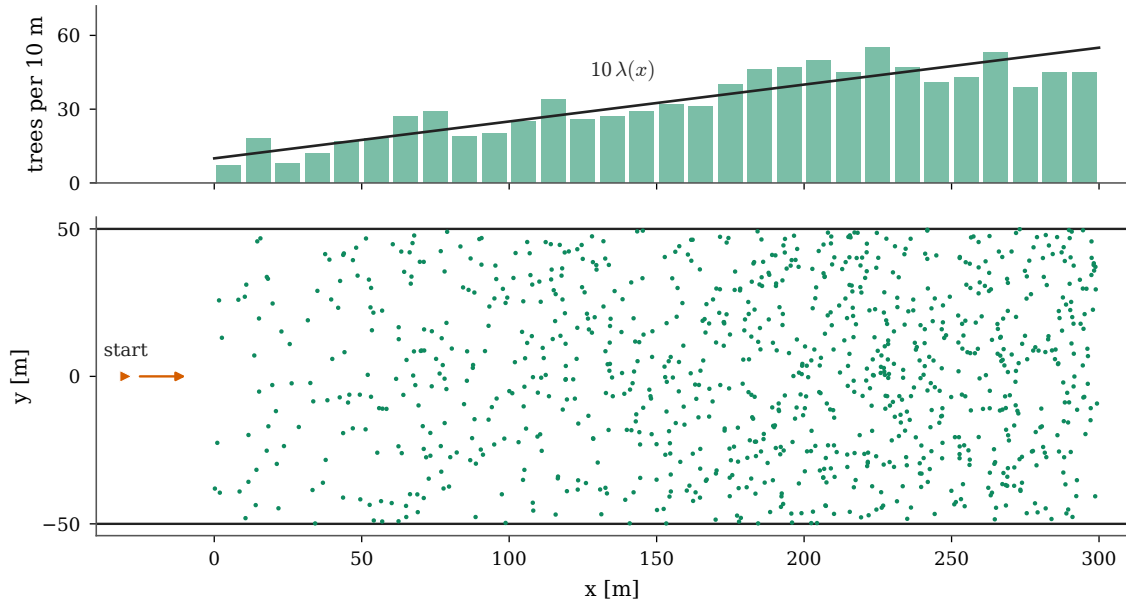


Figure 4.5: A growing density forest seen from above, with $\lambda_0 = 1$ and $\lambda_1 = 5.5$ trees per metre over 300 m, which gives the same number of trees, 975, as in Figure 4.4. The upper panel counts the trees in each 10 m of course and compares them with the profile (4.21).

The reason for this design is what it does to the measure. A growing forest is a course of increasing difficulty, and every flight, of whatever body, ends where the forest has become too dense for it: the progress of a flight reads directly as the density that the pair of body and controller can cope with. A single course therefore ranks the weak and the strong alike, the former in its first part and the latter in its last, and as long as the course is long enough that nobody reaches its end, the measure does not saturate. The role of chance shrinks as well. With the profile (4.21) the survival (4.20) of the blind straight flight becomes

$$S(x) = \exp \left[-\frac{w}{W} \left(\lambda_0 x + \frac{\lambda_1 - \lambda_0}{2L} x^2 \right) \right], \quad (4.23)$$

whose quadratic term makes long flights rapidly improbable instead of merely rare: the risk now grows with the distance, and flights end in a band rather than anywhere. In the forest of Figure 4.5 the blind flight covers 26 m on average and gets past 100 m less than once in a hundred attempts. Whatever a drone achieves beyond that is owed to what it sees and to how well its body lets it act on it, which is exactly what the progress is meant to measure.

The design also helps learning. A controller at the beginning of its training crashes early and therefore only ever experiences the sparse part of the forest; as it improves it reaches denser stretches, and the difficulty of its experience grows with its competence without any schedule having to be imposed. Finally, the positions remain independent, so a growing forest keeps the irregular texture of the uniform one, with its thickets and its clearings, and this turned out to matter, as the next subsection explains.

4.2.3 Latin Hypercube Forests

Independent positions have a cost. Nothing prevents two trunks from being drawn on top of each other, or a few of them from lining up so closely that they form a wall. The effect is not marginal. For independent positions of areal density n , the probability that a tree has a neighbour closer than a distance d is $1 - e^{-n\pi d^2}$: at one tree every 25 m², a density that the forests of this work reach in their last part, a quarter of the trunks overlap another trunk ($d = 2r$), and almost two thirds are separated from their nearest neighbour by a gap narrower than the wingspan of the reference drone ($d = 2r + b$). Chains of such pairs form barriers several metres wide (Figure 4.6, left). A flight that ends against one of them says little about the body or about the controller, and this luck of the draw is one of the sources of the noise that every comparison in this thesis has to average out.

The third generator was written to remove this effect. It borrows the idea of *Latin hypercube sampling* [30], the classical device for spreading random samples evenly over a domain: the domain is divided into strata and the samples are forced to occupy them evenly, while remaining random inside them. Here the corridor is tiled with rectangular cells and exactly one tree is placed in each cell, at a position uniform within it. To keep the density growing along the course, the cells shrink: the course is cut into columns whose width decreases linearly from s_0 at the entrance to s_1 at the end (the widths being rescaled so that they tile the length L exactly), and each column is cut into equal rows whose height follows the same interpolation (Figure 4.7). The name is used loosely. In a Latin hypercube proper each row and each column of the grid holds exactly one sample; here every cell does, which is the stronger form of stratification also known as jittered sampling.

The stratification does what it was meant to do. At the same density the overlapping trunks fall from 25% to 6% and the gaps narrower than the wingspan from 64% to 39% (Figure 4.6, right); no region can be crowded, because at most four trees can meet around the corner shared by four cells, and none can be empty, because every cell is occupied. The number of trees is also identical in all the forests of a batch, which removes the need for padding.

It was nevertheless abandoned, because what it removes is part of the task. A

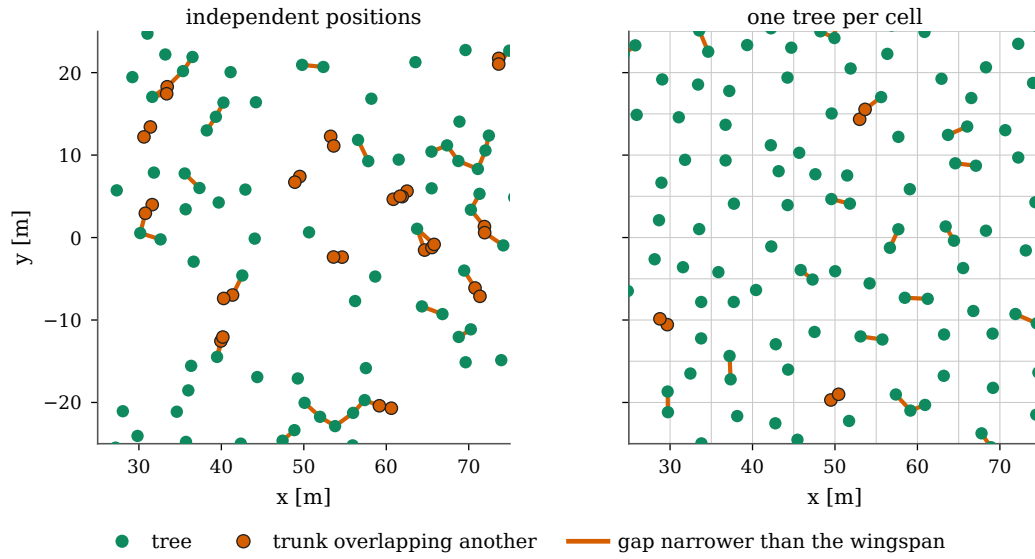


Figure 4.6: Independent and stratified positions at the same density, one tree every 25 m^2 , in a window of 50 m by 50 m with trunks to scale. With independent positions 25% of the trunks overlap another and 64% leave a gap narrower than the 1.4 m wingspan of the reference drone; with one tree per cell the two figures fall to 6% and 39% (averages over 200 forests).

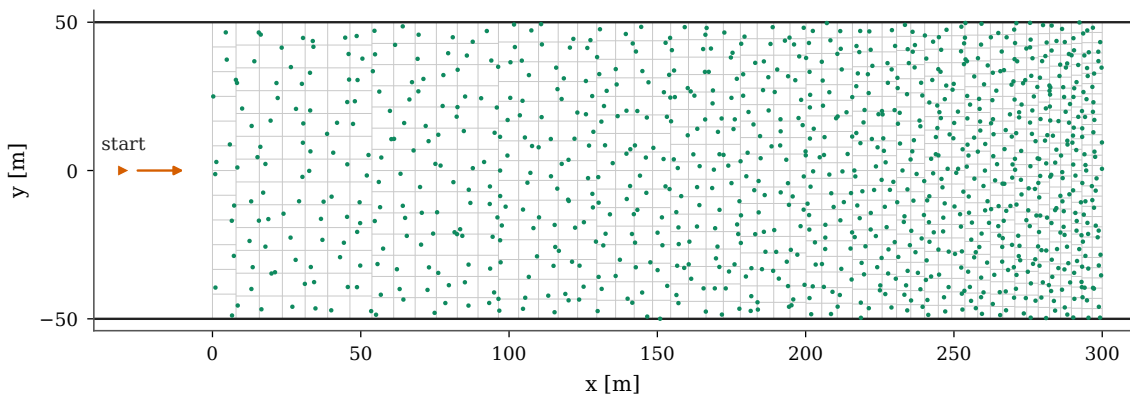


Figure 4.7: A Latin hypercube forest seen from above, with its cells. The cells shrink from 8 m to 3.4 m along the course and each holds exactly one tree, for 988 trees in total, about as many as in Figure 4.4 and Figure 4.5.

forest drawn in this way is tidy: the trees are almost evenly spaced, every gap is about one cell wide, and the thickets and the clearings that a real forest has, and that Figure 4.4 and Figure 4.5 show, are gone. A body and a controller selected in such a forest are selected in the presence of a regularity that no real environment offers, and a search could come to rely on it. Deciding when to go around a thicket and when to cut through a clearing is, on the contrary, part of what flying through a forest means. The choice made in this thesis is therefore to keep the forest natural and to pay for its luck with numbers: the results are obtained in growing density forests with independent positions, and the chance of an unlucky forest is averaged out over many of them.

4.3 Drone Model

The body that flies through the forest is a small electric aircraft with a morphing wing. It is not a fixed model: every body is generated by a script that turns a genome of fifteen numbers into a URDF file, with its geometry, its masses and inertias, its joints and the reference frames of the aerodynamic solver of subsection 4.1.4. What all bodies have in common is their architecture, that is, which parts they are made of, how these are actuated and what the drone can sense; what distinguishes them is the size, the proportions and the placement of those parts. This section describes the common architecture first, then the genome, and finally the particular body that is used as the reference throughout the thesis.

4.3.1 Drone Shape and Actuators

The drone has a conventional layout (Figure 4.8). A straight fuselage of constant cross-section, 10 cm wide, carries a propeller of 10 cm radius at its nose, a wing made of two independent rectangular panels attached to its sides, and a tail at its rear end formed by a horizontal surface, the elevator, and a vertical one, the rudder. The fuselage also houses the battery, the electronics and a small camera at the nose, which are fixed masses.

What is not conventional is the way the drone is controlled. There are no ailerons, no flaps and no hinged control surfaces of any kind: the surfaces move as a whole. Each wing panel is connected to the fuselage by two revolute joints in series that share the same point at its root, on the quarter-chord line. The first rotates the panel about the vertical axis and changes its *sweep*; the second rotates it about its own spanwise axis and changes its *twist*, that is, its incidence. The elevator is a single all-moving surface that pitches about a hinge at the end of the fuselage, and the rudder is an all-moving fin that turns about a vertical axis mounted on the elevator, so that it pitches with it. Together with the throttle of the propeller, this gives the seven commands that a controller issues at every control step:

$$a = (u, \delta_L^{sw}, \delta_R^{sw}, \delta_L^{tw}, \delta_R^{tw}, \delta^{el}, \delta^{ru}), \quad (4.24)$$

the throttle $u \in [0, 1]$ and six target angles, each bounded by the mechanical range of its joint. The two panels are commanded independently, so the wing can be

moved symmetrically, to trim the aircraft and to change its lift, or asymmetrically, to roll and to turn: a differential twist does the work of the ailerons of a conventional aircraft. This is the kind of morphing wing studied for winged drones by Bergonti et al. [3], here combined with a moving tail.

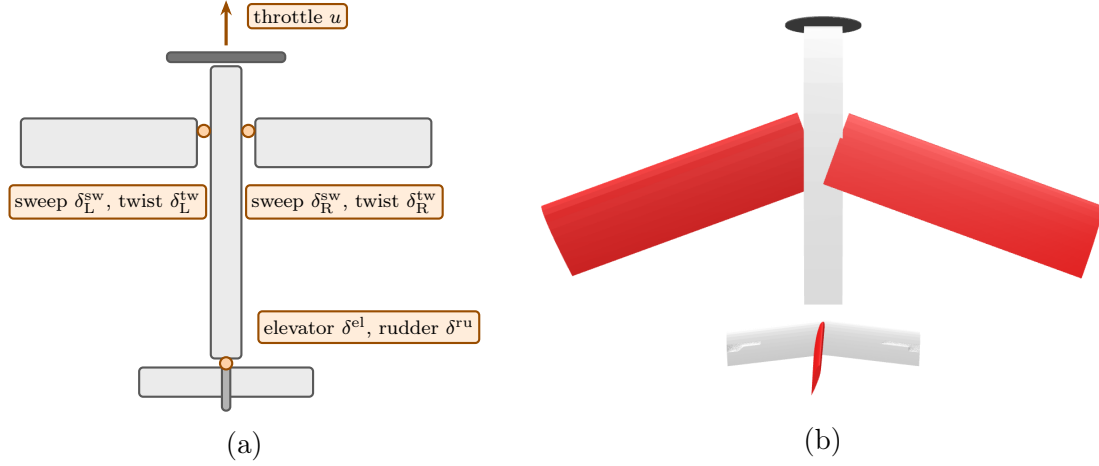


Figure 4.8: Parts and actuators of the drone, seen from above with the nose up. (a) The seven commands: the throttle of the propeller, the sweep and the twist of each wing panel, the elevator and the rudder. (b) The reference drone with both panels swept back to the limit of their range, twisted in opposite directions, and with the elevator and the rudder deflected.

Servomotors and gear ratios. Every joint is driven by a servomotor, modelled as in subsection 4.1.3: a position controller with a bounded torque, acting against the damping and the friction of the joint. The sweep joints use a larger servomotor than the twist and the tail joints (25 g against 10 g). Between the servomotor and a wing joint there is a reduction gear whose ratio k is part of the genome, one ratio for the sweep of both panels and one for their twist. A gear trades range for strength: the panel can move within

$$|\delta^{sw}| \leq \frac{0.7}{k_{sw}} \text{ rad}, \quad |\delta^{tw}| \leq \frac{0.5}{k_{tw}} \text{ rad}, \quad (4.25)$$

with a torque limit of $0.75 k_{sw}$ and $0.6 k_{tw}$ N m respectively, while the damping and the friction felt at the joint grow with k^2 and with k . With the ratios allowed by the genome, between 1.5 and 3.5, the sweep range goes from $\pm 27^\circ$ to $\pm 11^\circ$ and the twist range from $\pm 19^\circ$ to $\pm 8^\circ$. A body can thus have a wing that moves widely but weakly, or narrowly but firmly, and which of the two is better depends on the aerodynamic loads that its panels generate, that is, on the rest of the body. The tail joints have no gear: their range is ± 0.25 rad ($\pm 14^\circ$) and their torque limit 1 N m on every body. The controller never commands an angle directly: its six outputs lie in $[-1, 1]$ and are scaled by the range of the corresponding joint, so that the same output means “full deflection” on every body.

Masses. The structure is foam. The fuselage is a shell 15 mm thick and the wing and tail surfaces are solid panels whose thickness follows their airfoil section, all with a density of 20 kg/m³ (reduced by a fill factor of 0.68 for the surfaces); each part receives the mass of its volume and the inertia tensor of a box of its dimensions. To this are added the fixed masses: 200 g for the battery and the electronics, 30 g for the camera, 41.5 g for the motor and the propeller, 90 g for the six servomotors and 40 g for hinges and fittings. The fixed masses amount to about 0.40 kg and are the same for every body; the structural mass is what the genome controls. It grows quickly with the size of the wing, since the volume of a panel is proportional to the cube of its span for a given aspect ratio and section: the lightest body that the genome can express weighs 0.44 kg and the heaviest 2.5 kg, while nine bodies out of ten drawn uniformly at random weigh between 0.50 and 1.10 kg. The position of the centre of mass is controlled by a gene as well, which places the mass of the fuselage, battery included, along its length.

Energy. The second quantity by which a body is judged, next to the progress of section 4.2, is the energy it spends. The power absorbed by the propeller follows from its rotation speed n and its advance ratio J , with the same kind of law as the thrust (4.16):

$$P_{\text{prop}} = \rho n^3 d^5 C_{P_0} \max(0, 1 - c'_1 J - c'_2 J^2), \quad (4.26)$$

with $C_{P_0} = 0.044$, $c'_1 = 0$ and $c'_2 = 1.104$, which gives about 83 W at full throttle with the drone at rest. Each servomotor absorbs the mechanical power it delivers plus a resistive loss proportional to the square of its torque, and never less than zero, since a servomotor that is driven backwards by the air does not recharge the battery. Holding a panel against a steady aerodynamic load therefore costs energy even when nothing moves. No other consumption is modelled, and the losses of the motor and of its speed controller are neglected. The energy E of a flight is the integral of the total power up to the end of the flight, and it is compared across bodies of different weight through the dimensionless *cost of transport*,

$$CoT = \frac{E}{m g \Delta x}, \quad (4.27)$$

where m is the nominal mass of the body, g the acceleration of gravity and Δx the progress of the flight: the energy spent to carry one unit of weight over one unit of distance along the corridor. Distance flown sideways or in circles adds to the energy and not to the progress, so a body that needs wide manoeuvres to get through the trees pays for them.

4.3.2 Sensors

Every body carries the same sensors, and their readings are the only information about the world that a controller receives. Table 4.1 lists them. The drone knows its altitude, its attitude and its velocity, and it has a depth sensor that looks ahead. It does not know where it is along or across the corridor, it has no measure of its angular rates, of its airspeed or of the angles of its own joints, and no sensor tells it which body it is flying. The measurements are delivered at the control rate of 25 Hz

and each is corrupted by Gaussian noise, as anticipated in subsection 4.1.5. Besides the measurements, the controller receives two quantities that are not sensed: the seven commands it issued at the previous step, and the forward speed it is asked to keep.

Quantity	Size	Noise (standard deviation)
Altitude above the ground	1	0.1 m
Attitude, as a unit quaternion	4	0.01 on each component
Linear velocity in the world frame	3	0.4 m/s along the corridor, 0.2 m/s on the other axes
Depth ahead, one value per sector	20	1.5 m
Commands issued at the previous step	7	none
Commanded forward speed	1	none

Table 4.1: What the controller receives at every control step: 28 measurements, with the noise added to each, and 8 quantities that are not measured, for 36 numbers in total.

Depth sensor. The depth sensor is what lets the drone see the forest. It looks forward in the horizontal plane: a fan of 80° centred on the heading is divided into 20 equal sectors, and each sector reports the distance to the surface of the nearest trunk that falls inside it, even partially, or to the lateral wall of the corridor if that is closer, up to a range of 30 m (Figure 4.9). When the drone banks, the fan narrows by the cosine of the roll angle, as the horizontal footprint of a sensor fixed to the body would. The result is a one-dimensional depth image of 20 pixels, a deliberately poor sensor: it has no vertical resolution, its range is about two seconds of flight, and a trunk hidden behind another is invisible until the first has been passed. The sensor is one of the custom Taichi kernels mentioned in subsection 4.1.2: one GPU thread for each pair of environment and sector, with an inner loop over the trees of that environment.

4.3.3 Drone Genome

A body is a point in a space of fifteen parameters, its *genome* (Table 4.2). Each gene is stored as a number in $[0, 1]$, the form in which the evolutionary algorithms handle it, and is mapped linearly onto the physical range of its parameter when the body is built. Twelve genes are continuous. The remaining three select the wing section among the four-digit NACA airfoils and are discrete: the unit interval is

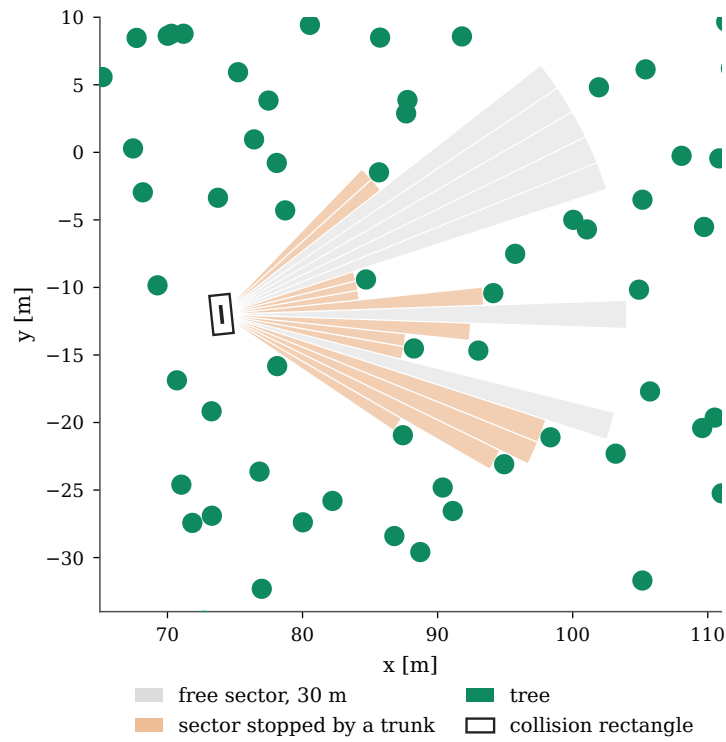


Figure 4.9: What the forest is to the drone, seen from above. The depth sensor divides a fan of 80° ahead of the drone into 20 sectors and reports for each the distance to the nearest trunk, up to 30 m. The collision test of section 4.2 asks whether the centre of a trunk lies inside the rectangle drawn around the wing (trunks and wingspan are to scale).

divided into as many equal bins as there are admissible values, and the gene takes the value of the bin it falls in.

The first two genes define the wing. Its panels are rectangular, and the genes give the span b_p of one panel and its aspect ratio, from which the chord is $\bar{c} = b_p / AR_p$. The wing as a whole is what these numbers imply once the two panels are mounted on the sides of a fuselage of width $w_f = 0.1$ m: a span of $2b_p + w_f$ from tip to tip, between 0.9 and 1.9 m, and an aspect ratio of $(2b_p + w_f) / \bar{c}$, which is the one that enters the aerodynamic model (subsection 4.1.4) and which for the reference drone is 7.5. The same construction holds for the elevator, whose gene is its total span, while the rudder is a single panel. Genes 3 to 5 lay the aircraft out along its axis: the length of the fuselage sets the lever arm of the tail, which sits at its end, and the positions of the wing and of the mass of the fuselage along it decide, together, how far the centre of mass lies from the point where the lift of the wing acts. This distance governs the longitudinal stability of an aircraft, and a small change of either gene can turn a docile body into one that is hard or impossible to control. The dihedral gene tilts the two panels upwards or downwards at their root, and the two gear ratios set the authority of the morphing wing, as described above.

The airfoil genes act in two ways. The thickness of the section is the thickness of the panel, and therefore changes its mass and its inertia. And the section as a whole selects a row in the table of airfoil parameters of subsection 4.1.4, from which the aerodynamic solver reads the zero-lift angle, the stall angle, the parasitic drag and the nominal Reynolds number of the wing. The tail surfaces are not part of the genome in this respect: they always use the same symmetric section, 16% thick.

No constraint is imposed on the genome beyond its ranges. The generator does not check that a body is well proportioned, that its wing and its tail do not interfere, or that it is stable: any point of the unit hypercube becomes a body, and whether that body can fly is left for the flights to decide. The ranges were chosen so that a large part of the space is flyable, but nothing prevents the search from visiting the rest.

4.3.4 The Standard Drone

One body is singled out as the reference of the whole thesis (Figure 4.10): it is the body against which the evolved ones are compared, and the point from which the quantities of this chapter were illustrated. Its genome is the last column of Table 4.2. Its wing measures 1.5 m from tip to tip with a chord of 0.2 m and a NACA 3416 section, its fuselage is 0.73 m long for an overall length of 0.89 m, and it weighs 0.605 kg.

These numbers are not arbitrary. The dimensions of the standard drone and the section of its wing are those of the H-King Bixler 3 [22], a commercial foam glider with a wingspan of 1.55 m and a length of 0.95 m that is widely used as a research platform, and that Bergonti et al. [3] also adopt as the baseline of their co-design study. For this reason the figures of this thesis label the standard drone with the name of that aircraft. The correspondence, however, stops at the geometry. The Bixler 3 is flown with ailerons, an elevator and a rudder hinged on fixed surfaces, and is pushed by a propeller mounted behind its wing; the standard drone has none

	Gene	Range	Reference drone
1	Span of one wing panel	0.4 to 0.9 m	0.7 m
2	Aspect ratio of one wing panel (span over chord)	1.5 to 5	3.5
3	Length of the fuselage	0.4 to 0.9 m	0.73 m
4	Position of the mass of the fuselage, as a fraction of its length from the nose	0.30 to 0.60	0.38
5	Position of the wing root, as a fraction of the fuselage length from the nose	0.30 to 0.60	0.38
6	Span of the elevator	0.2 to 0.6 m	0.5 m
7	Aspect ratio of the elevator	1.5 to 4	4
8	Span of the rudder	0.1 to 0.4 m	0.2 m
9	Aspect ratio of the rudder	1.5 to 4	2
10	Dihedral angle of the wing panels	-4° to 4°	0°
11	Gear ratio of the sweep joints	1.5 to 3.5	2
12	Gear ratio of the twist joints	1.5 to 3.5	2.5
13	Airfoil: maximum camber (first NACA digit)	0 to 4	3
14	Airfoil: position of the maximum camber (second digit)	2 to 5	4
15	Airfoil: thickness (last two digits)	10 to 20	16

Table 4.2: The genome of the drone: fifteen genes, their physical ranges, and the values of the reference drone of subsection 4.3.4. The last three genes are discrete.

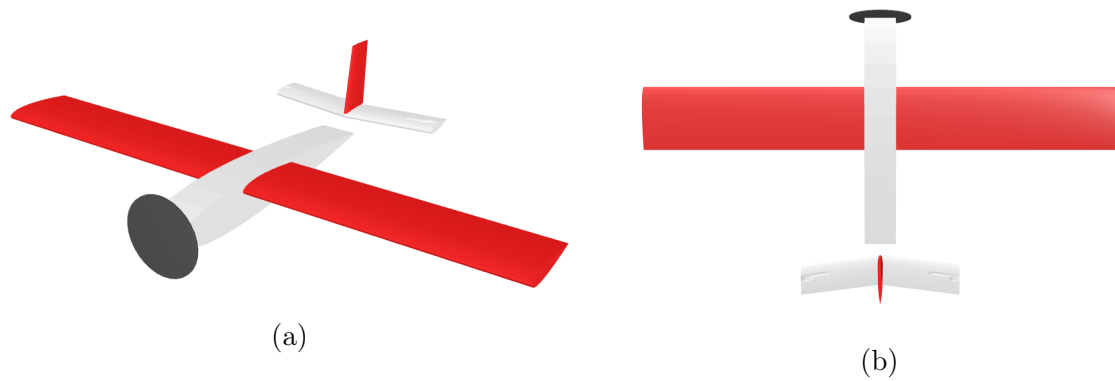


Figure 4.10: The standard drone with all joints in their neutral position: (a) three-quarter view, (b) top view with the nose up. The wing panels and the rudder are drawn in red, the fuselage and the elevator in grey, the disc of the propeller in black.

of those control surfaces, steers by sweeping and twisting its wing panels and by moving its tail surfaces as a whole, and is pulled by a propeller at its nose. It is also lighter, since the commercial aircraft weighs 0.89 kg ready to fly. The two aircraft share a shape, not a set of actuators, and their flight dynamics are therefore not directly comparable: the standard drone should be read as a body of familiar proportions inside the design space of Table 4.2, not as a model of the commercial aircraft.

Methodology

5.1 Base Controllers

5.1.1 Specialist LSTM Controller

5.1.2 Generalist LSTM Controller

5.1.3 Training the Generalist LSTM Controller

5.2 The Hebbian Controller

5.2.1 Hebbian Rules on the Output Layer

5.2.2 Weight Decay and Reset Semantics

5.3 Inner Loop: Evolving the Hebbian Rules

5.4 Outer Loop: Evolving the Drone Morphology

5.5 The Full Co-Design Pipeline

Results and Experiments

6.1 Inner Loop Experiments

6.1.1 Recovering Performance on Generalist Controllers

6.1.2 Improving Performance on Specialist Controllers

6.2 Full Co-Design Experiments

6.2.1 Co-Design vs. Morphology-Only Evolution

6.2.2 Swapping the Pareto Fronts

6.2.3 PCA Analysis of the Evolved Morphologies

6.2.4 Morphology Choices of The Evolution

6.2.5 Generalization Capabilities of Evolved Controllers

6.2.6 Convergence or Plasticity?

Conclusions & Future Work

7.1 Conclusions

7.2 Future Work

7.2.1 Meta-Learning Techniques

7.2.2 Testing on Real Drones

7.2.3 Testing on Different Systems

List of Figures

2.1	The agent-environment interaction loop	7
2.2	Actor and critic networks	10
2.3	A LSTM gate	13
2.4	Structure of an LSTM cell	14
2.5	Covariance adaptation in CMA-ES	16
2.6	Pareto dominance, non-dominated fronts and crowding distance . . .	18
2.7	Locality of the Hebbian update	20
4.1	Life of a Taichi kernel	32
4.2	One control step of the simulation	36
4.3	Lift and drag coefficients of the aerodynamic model	39
4.4	A uniform density forest	44
4.5	A growing density forest	45
4.6	Independent and stratified positions at the same density	47
4.7	A Latin hypercube forest	47
4.8	Parts and actuators of the drone	49
4.9	Depth sectors and collision rectangle	52
4.10	The standard drone	55

Bibliography

- [1] John D. Anderson. *Fundamentals of Aerodynamics*. 6th ed. New York, NY: McGraw-Hill Education, 2017.
- [2] Randal W. Beard and Timothy W. McLain. *Small Unmanned Aircraft: Theory and Practice*. Princeton, NJ: Princeton University Press, 2012.
- [3] Fabio Bergonti et al. “Co-Design Optimisation of Morphing Topology and Control of Winged Drones”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2024, pp. 8679–8685.
- [4] Nick Cheney et al. “Scalable Co-Optimization of Morphology and Control in Embodied Machines”. In: *Journal of The Royal Society Interface* 15.143 (2018), p. 20170937.
- [5] Kalyanmoy Deb and Ram Bhushan Agrawal. “Simulated Binary Crossover for Continuous Search Space”. In: *Complex Systems* 9.2 (1995), pp. 115–148.
- [6] Kalyanmoy Deb et al. “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”. In: *IEEE Transactions on Evolutionary Computation* 6.2 (2002), pp. 182–197.
- [7] Fuda van Diggelen et al. *Emergent Heterogeneous Swarm Control Through Hebbian Learning*. <https://arxiv.org/abs/2507.11566>. 2025.
- [8] Mark Drela. “XFOIL: An Analysis and Design System for Low Reynolds Number Airfoils”. In: *Low Reynolds Number Aerodynamics*. Ed. by Thomas J. Mueller. Vol. 54. Lecture Notes in Engineering. Berlin, Heidelberg: Springer, 1989, pp. 1–12. DOI: 10.1007/978-3-642-84010-4_1.
- [9] Agoston E. Eiben and Emma Hart. “If It Evolves It Needs to Learn”. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO '20)*. ACM, 2020, pp. 1383–1384.
- [10] Agoston E. Eiben and James E. Smith. *Introduction to Evolutionary Computing*. 2nd ed. Springer, 2015.
- [11] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211.
- [12] Roy Featherstone. *Rigid Body Dynamics Algorithms*. New York, NY: Springer, 2008. DOI: 10.1007/978-1-4899-7560-7.
- [13] Andrea Ferigo et al. “Evolving Hebbian Learning Rules in Voxel-Based Soft Robots”. In: *IEEE Transactions on Cognitive and Developmental Systems* 15.3 (2023), pp. 1536–1546.

- [14] Dario Floreano and Joseba Urzelai. “Evolutionary Robots with On-line Self-Organization and Behavioral Fitness”. In: *Neural Networks* 13.4–5 (2000), pp. 431–443.
- [15] Genesis Authors. *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. <https://github.com/Genesis-Embodied-AI/Genesis>. Dec. 2024.
- [16] Wulfram Gerstner and Werner M. Kistler. “Mathematical Formulations of Hebbian Learning”. In: *Biological Cybernetics* 87.5–6 (2002), pp. 404–415.
- [17] Agrim Gupta et al. “Embodied Intelligence via Learning and Evolution”. In: *Nature Communications* 12 (2021), p. 5721.
- [18] Agrim Gupta et al. “MetaMorph: Learning Universal Controllers with Transformers”. In: *International Conference on Learning Representations (ICLR)*. 2022.
- [19] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. <https://arxiv.org/abs/1604.00772>. 2016.
- [20] Nikolaus Hansen and Andreas Ostermeier. “Completely Derandomized Self-Adaptation in Evolution Strategies”. In: *Evolutionary Computation* 9.2 (2001), pp. 159–195.
- [21] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.
- [22] HobbyKing. *H-King Bixler 3 (PNF) Glider 1550mm (61")*. Product page, https://hobbyking.com/en_us/h-king-bixler-3-glider-1550-pnf.html. Accessed 18 September 2026. 2026.
- [23] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [24] Yuanming Hu et al. “Taichi: A Language for High-Performance Computation on Spatially Sparse Data Structures”. In: *ACM Transactions on Graphics* 38.6 (2019), 201:1–201:16. DOI: 10.1145/3355089.3356506.
- [25] Yuanming Hu et al. “DiffTaichi: Differentiable Programming for Physical Simulation”. In: *International Conference on Learning Representations (ICLR)*. 2020.
- [26] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. “Planning and Acting in Partially Observable Stochastic Domains”. In: *Artificial Intelligence* 101.1–2 (1998), pp. 99–134.
- [27] Vijay R. Konda and John N. Tsitsiklis. “Actor-Critic Algorithms”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2000.
- [28] Hod Lipson and Jordan B. Pollack. “Automatic Design and Manufacture of Robotic Lifeforms”. In: *Nature* 406.6799 (2000), pp. 974–978.
- [29] Barnes W. McCormick. *Aerodynamics, Aeronautics, and Flight Mechanics*. 2nd ed. New York, NY: John Wiley & Sons, 1995.

- [30] Michael D. McKay, Richard J. Beckman, and William J. Conover. “A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code”. In: *Technometrics* 21.2 (1979), pp. 239–245. DOI: 10.1080/00401706.1979.10489755.
- [31] Thomas Miconi, Kenneth O. Stanley, and Jeff Clune. “Differentiable Plasticity: Training Plastic Neural Networks with Backpropagation”. In: *International Conference on Machine Learning (ICML)*. 2018, pp. 3559–3568.
- [32] Thomas Miconi et al. “Backpropamine: Training Self-Modifying Neural Networks with Differentiable Neuromodulated Plasticity”. In: *International Conference on Learning Representations (ICLR)*. 2019.
- [33] Jed R. Muff et al. “Unconventional Hexacopters via Evolution and Learning: Performance Gains and New Insights”. In: *Applications of Evolutionary Computation (EvoApplications 2026)*. Vol. 16524. Lecture Notes in Computer Science. Preprint: arXiv:2505.14129. Springer, 2026.
- [34] Elias Najjarro and Sebastian Risi. “Meta-Learning through Hebbian Plasticity in Random Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 20719–20731.
- [35] Yael Niv et al. “Evolution of Reinforcement Learning in Uncertain Environments: A Simple Explanation for Complex Foraging Behaviors”. In: *Adaptive Behavior* 10.1 (2002), pp. 5–24.
- [36] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. In: *International Conference on Machine Learning (ICML)*. 2013.
- [37] Joachim Winther Pedersen and Sebastian Risi. “Evolving and Merging Hebbian Learning Rules: Increasing Generalization by Decreasing the Number of Rules”. In: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO ’21)*. ACM, 2021.
- [38] Xue Bin Peng et al. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528.
- [39] Tim Salimans et al. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. <https://arxiv.org/abs/1703.03864>. 2017.
- [40] John Schulman et al. “Trust Region Policy Optimization”. In: *International Conference on Machine Learning (ICML)*. 2015.
- [41] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations (ICLR)*. 2016.
- [42] John Schulman et al. *Proximal Policy Optimization Algorithms*. <https://arxiv.org/abs/1707.06347>. 2017.
- [43] Karl Sims. “Evolving Virtual Creatures”. In: *Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH ’94)*. ACM, 1994, pp. 15–22.

- [44] Andrea Soltoggio, Kenneth O. Stanley, and Sebastian Risi. “Born to Learn: The Inspiration, Progress, and Future of Evolved Plastic Artificial Neural Networks”. In: *Neural Networks* 108 (2018), pp. 48–67.
- [45] Andrea Soltoggio et al. “Evolutionary Advantages of Neuromodulated Plasticity in Dynamic, Reward-Based Scenarios”. In: *Proceedings of the 11th International Conference on Artificial Life (ALIFE XI)*. MIT Press, 2008, pp. 569–576.
- [46] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018.
- [47] Richard S. Sutton et al. “Policy Gradient Methods for Reinforcement Learning with Function Approximation”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 1999.
- [48] Stone Tao. *How fast is the new hyped Genesis simulator?* <https://stoneztao.substack.com/p/the-new-hyped-genesis-simulator-is>. Dec. 2024.
- [49] Josh Tobin et al. “Domain randomization for transferring deep neural networks from simulation to the real world”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30. DOI: 10.1109/IROS.2017.8202133.
- [50] Emanuel Todorov. “Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2014, pp. 6054–6061. DOI: 10.1109/ICRA.2014.6907751.
- [51] Emanuel Todorov, Tom Erez, and Yuval Tassa. “MuJoCo: A physics engine for model-based control”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.
- [52] Christopher J. C. H. Watkins and Peter Dayan. “Q-Learning”. In: *Machine Learning* 8.3–4 (1992), pp. 279–292.
- [53] Paul J. Werbos. “Backpropagation Through Time: What It Does and How to Do It”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560.
- [54] Eckart Zitzler and Lothar Thiele. “Multiobjective Evolutionary Algorithms: A Comparative Case Study and the Strength Pareto Approach”. In: *IEEE Transactions on Evolutionary Computation* 3.4 (1999), pp. 257–271.